Book Finder

Introduction:

It is an app where a book recommender suggests books to readers. It helps readers discover, explore, and find books tailored to their preferences. Users can browse through curated lists and explore books that are trending within their preferred genres. The app aims to create an engaging, interactive, and personalized reading experience.

Users of the System:

1. BookRecommender
2. BookReader

BookRecommender Actions:

BookRecommender will suggest the books to read.

BookReader Actions:

BookReader will view the list of books created.

Modules of the Application:

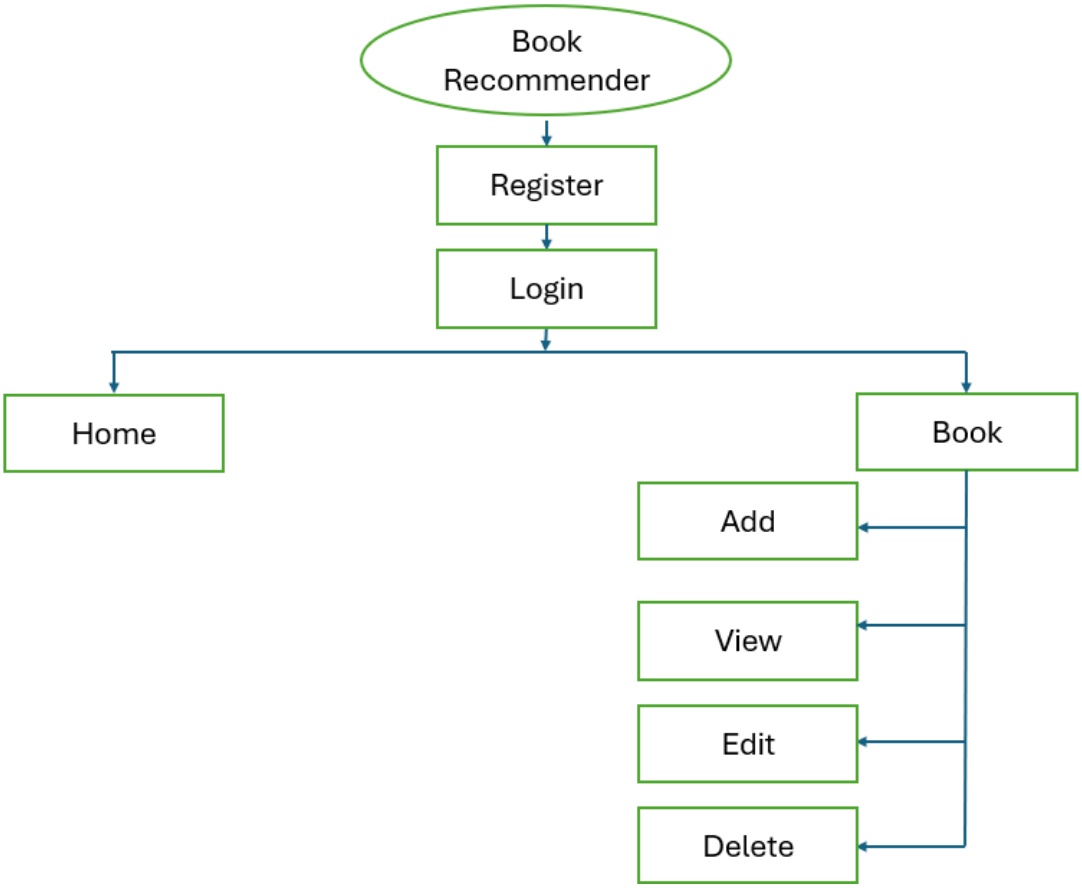
BookRecommender:

- Register
- Login
- Home
    - Add Book
    - View Book Details
    - Edit Book
    - Delete Book

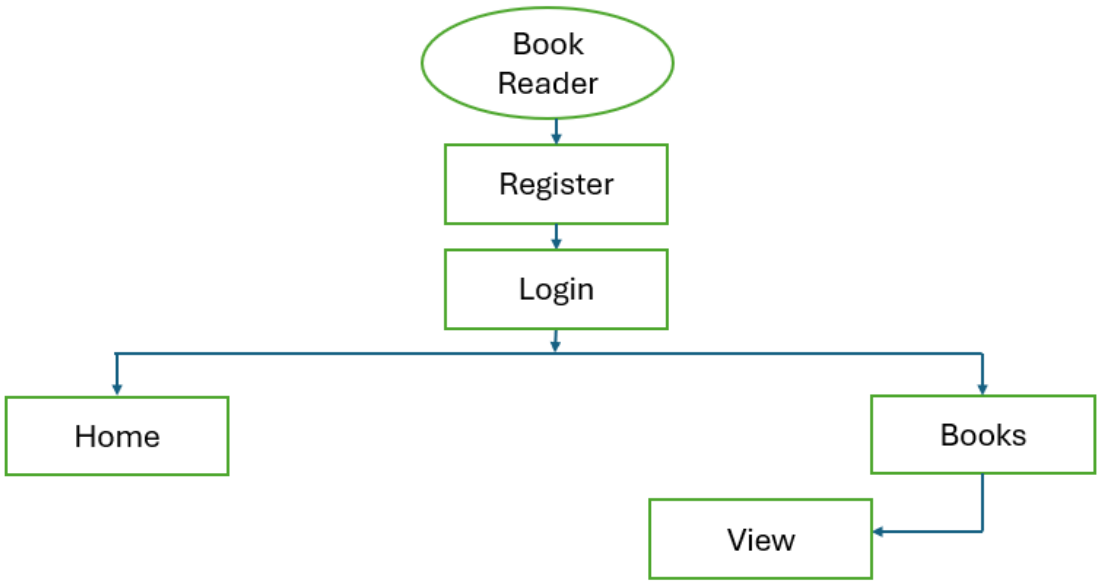
BookReader:

- Register
- Login
- Home
    - View books.

BookRecommender Flow Diagram:



BookReader Flow Diagram:



Technology Stack

Front End

React

Back End

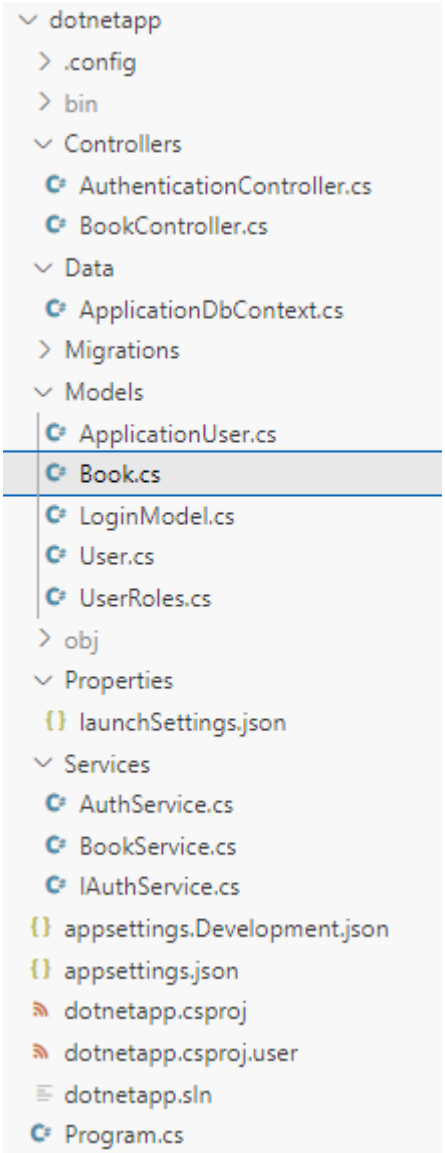
.NET Web API, EF Core, Microsoft SQL Server Database.

Application assumptions:

1. The login page should be the first page rendered when the application loads.
2. Manual routing should be restricted by implementing Auth Guard, utilizing the canActivate interface. For example, if the user enters as http://localhost:8080/dashboard or http://localhost:8080/user the page should not navigate to the corresponding page instead it should redirect to the login page.
3. Unless logged into the system, the user cannot navigate to any other pages.
4. Logging out must again redirect to the login page.

Backend Requirements:

Create folders named as model, controller, service and configuration inside dotnetapp.



<u>ApplicationDbContext</u>: (/Data/ApplicationDbContext.cs)

Namespace: dotnetapp.Data

Inside **Data** folder create **ApplicationDbContext** file with the following DbSet mentioned below

```
public DbSet<Book> Books { get; set; }
public  DbSet<User> Users { get; set; }
```

<u>Model Classes:</u>

Inside **Models** folder create all the model classes mentioned below.
Namespace: dotnetapp.Models

User (Models / User.cs):

This class stores the user role (Book Recommender or Reader) and all user information.

Properties:

- UserId: int
- Email: string
- Password: string
- Username: string
- MobileNumber: string
- UserRole: string (BookRecommender/Reader)

Book (Models / Book.cs):

This class stores information about a Book.

Properties:

- BookId: int
- Title: string
- Author: string
- Genre: string
- PublishedDate: string
- CoverImage: string

LoginModel (Models / LoginModel.cs):

This class stores the email and password to authenticate the user during login.

Properties:

- Email: string
- Password: string

UserRoles (Models / UserRoles.cs):

This class defines constants for user roles.

Constants:

1. BookRecommender: string
2. BookReader: string

ApplicationUser (Models / ApplicationUser.cs):

This class represents a user in the application, inheriting from **IdentityUser** class.

Property:

- Name: string (Max length 30)

Important note:

Implement database logic only in the service file functions **without using try-catch.** Use **try-catch** only in the controller files and call the service file functions inside it.

Services:

Inside "**Services"** folder, create all the services file mentioned below.

Namespace: dotnetapp.Services

BookService (Services/BookService.cs)

This service class provides methods to interact with Book Details stored in the database.

Constructor:

```
public BookService(ApplicationDbContext context)
    {
        _context = context;
    }
```

Functions:

1. **public async Task<IEnumerable<Book>> GetAllBooks():**
   a. Retrieves all books from the database.
   b. Returns a collection of books.

2. **public async Task<Book> GetBookById(int bookId):**
   a. Retrieves a book by its ID from the database.
   b. Returns the book object if found, otherwise it throws a **BookException** with the message "Cannot find any book".

3. **public async Task<bool> AddBook(Book book):**

   a. The method first checks if a book with the same title already exists in the database.
   b. If such a book exists, it throws a **BookException** with the message "**Failed to add book**".
   c. If no such book exists, it adds the new book to the database context.
   d. If a book is added, it should response with a message "Book added successfully".
   e. It then saves the changes to the database asynchronously.
   f. Finally, it returns true to indicate the book was successfully added.

4. **public async Task<bool> UpdateBook(int bookId, Book book):**
   a. Updates an existing book in the database.
   b. Retrieves the existing book by its ID.
   c. If the book is not found, returns false.
   d. Updates the existing book with the provided data.
   e. Saves changes asynchronously to the database.
   f. Returns true for successful update.

5. **public async Task<bool> DeleteBook(int bookId):**
   a. Deletes an existing book from the database.
   b. Retrieves the book by its ID.

c. If the book is not found, returns false.
d. Removes the book from the database.
e. If a book is deleted, it should display message “Book deleted successfully”.
f. Saves changes asynchronously to the database.
g. Returns true for successful deletion.

<u>AuthService</u> (Services / AuthService.cs):

The AuthService class is responsible for user authentication and authorization.

Functions:

1. **public async Task<(int, string)> Registration (User model, string role):**
    a. Registers a new user with the provided details and assigns a role.

2. **public async Task<(int, object)> Login(LoginModel model):**
    a. Find user by email in the database.
    b. Check if user exists, if not return “**Invalid email”.**
    c. If the user exists, check the password is correct, if not return “**Invalid password”.**
    d. Logs in a user with the provided credentials and generates a JWT token for authentication.

3. **private string GenerateToken(IEnumerable<Claim> claims):**
    a. Generates a JWT token based on the provided claims.

<u>IAuthService</u> (Services / IAuthService.cs):

The **IAuthService** is an interface that defines methods for user registration and login.

Methods:

1. Task< (int, string)> Registration (User model, string role);
2. Task< (int, object)> Login (LoginModel model);

<u>Controllers</u>:

Inside “**Controllers”** folder create all the controllers file mentioned below.
Namespace: dotnetapp.Controllers

AuthenticationController (Controllers / AuthenticationController.cs):

This controller handles user authentication and registration requests.

Functions:

1. **public async Task<IActionResult> Login(LoginModel model)**
    a. Accepts login requests, validates the payload, and calls the authentication service to perform user login. Returns a JWT token upon successful login.

2. **public async Task<IActionResult> Register(User model):**
    a. Accepts registration requests, validates the payload, and calls the authentication service to register a new user. Returns a success message upon successful registration.

BookController (Controllers/BookController.cs)
This controller manages books, interacting with the BookService to perform CRUD operations.

<u>Functions:</u>
1. public async Task<ActionResult<IEnumerable<Book>>> GetAllBooks():
a. Retrieves all books using the GetAllBooks method from the BookService.
b. Returns a 200 OK response with the list of books.

2. public async Task<ActionResult<Book>> GetBookById(int bookId):
a. Retrieves a book by its ID using the GetBookById method from the BookService.
b. If the book is not found, returns a 404 Not Found response with a message "Book not found".
c. Returns a 200 OK response with the book data.

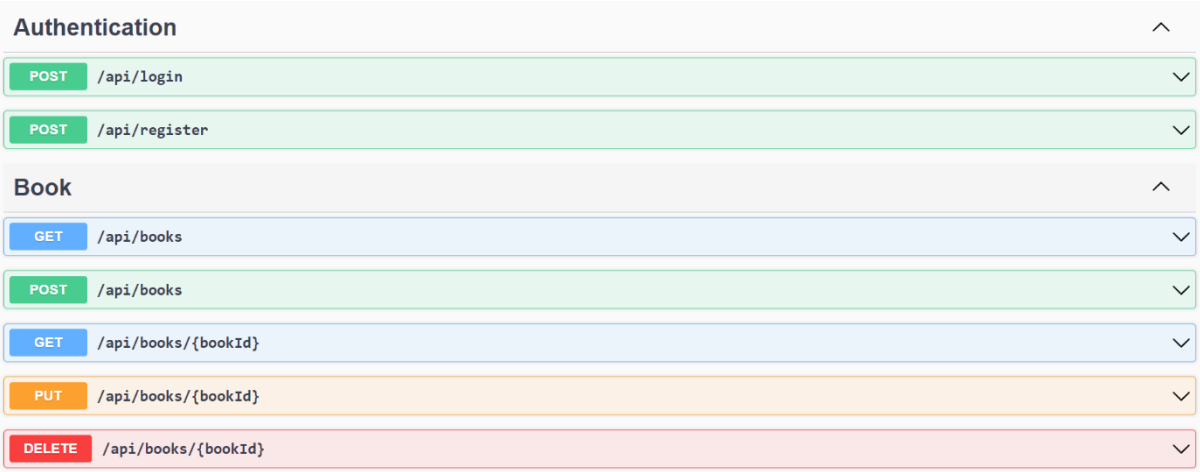
3. public async Task<ActionResult> AddBook([FromBody] Book book):
a. Adds a new book.
b. Receives the book details in the request body.
c. Tries to add the book using the AddBook method from the BookService.
d. If adding the book is successful, returns a 201 Created response with a success message "Book added successfully".
e. If adding the book fails, returns a 500 Internal Server Error response with a failure message "Failed to add book".
f. If an exception occurs during the process, returns a 500 Internal Server Error response with the exception message.

4. public async Task<ActionResult> UpdateBook(int bookId, [FromBody] Book book):
a. Updates an existing Book Details.
b. Receives the book ID and updated data in the request body.
c. Tries to update the book using the UpdateBook method from the BookService.
d. If the update is successful, returns a 200 OK response with a success message "Book updated successfully".
e. If the book is not found, returns a 404 Not Found response with a message "Cannot find any book".
f. If an exception occurs during the process, returns a 500 Internal Server Error response with the exception message.

5. public async Task<ActionResult> DeleteBook(int bookId):

a. Deletes an existing book.
b. Receives the book ID to be deleted.
c. Tries to delete the book using the Delete Book method from the BookService.
d. If the deletion is successful, returns a 200 OK response with a success message “Book deleted successfully”.
e. If the book is not found, returns a 404 Not Found response with a message “Cannot find any book”.
f. If an exception occurs during the process, returns a 500 Internal Server Error response with the exception message.

Endpoints:



Note:

1. Ensure JWT authentication is applied to all endpoints, with role-based access control:
   - The endpoint for adding a plant requires a valid JWT in the header. If not provided or invalid, return an Unauthorized error.
   - Only users with the " BookRecommender" role can access this endpoint. If a user with the "BookReader" role (or any other unauthorized role) attempts access, return a Forbidden error.

1. Login: [Access for both Book Recommender and Book Reader]

Endpoint name: “/api/login”
Method: POST
Request body: {
"Email": "string",
"Password": "string"

}

Response:

Status Code	Response body

201 (HttpStatusCode OK)	JSON object containing a JWT token.
	Example: {
	"Status": "Success",
	"token":
	"eyJhbGciOiJIUzI1NiiQWRtaW4iLCJqdGkiOiJmZTUyYzAxZS1"
	}

400 BadRequest	JSON object containing Error message.
500	JSON object containing Error message.

2. <u>Register</u>: [Access for both BookRecommender and BookReader]

Endpoint name: "/api/register"

Method: POST

Request body:

```
{
  "Username": "string",
  "Email": "user@example.com",
  "MobileNumber": "9876541221",
  "Password": "Pass@2425",
  "UserRole": "string"
}
```

Response:

Status Code	Response body
201 (HttpStatusCode OK)	JSON object containing success message.
400 BadRequest	JSON object containing Error message.
500	JSON object containing Error message.

3. <u>Get all Books:</u> [Access for only both users]

Endpoint name: "/api/ books"

Method: GET

Response:

Status Code	Response body
200 (HttpStatusCode OK)	JSON object containing all the books.
500	JSON object containing Error message.

4. <u>Add Book:</u> [Access for only BookRecommender]

Endpoint name: "/api/books"

Method: POST

Request body:

```
[
  {
    "BookId": 0,
    "Title": "string",
    "Author": "string",
    "Genre": "string",
    "PublishedDate": "string",
    "CoverImage": "string"
  }
]
```

Response:

Status Code	Response body
201 (HttpStatusCode OK)	JSON object containing success message.
400 BadRequest	JSON object containing Error message.
500	JSON object containing Error message.

5. <u>Get specific Book:</u> [Access for both Users]

Endpoint name: "/api/books/{bookId}"

Method: GET

Parameter: bookId

Response:

Status Code	Response body
200 (HttpStatusCode OK)	JSON object containing book details.
404 NotFound	JSON object containing Error message.
500	JSON object containing Error message.

6. <u>Update Book:</u> [Access for only BookRecommender]

Endpoint name: "/api/books /{bookId}"

Method: PUT

Parameter: bookId

Request body:

```
{
  "BookId": 0,
  "Title": "string",
  "Author": "string",
  "Genre": "string",
  "PublishedDate": "string",
  "CoverImage": "string"
}
```

Response:

Status Code	Response body
200 (HttpStatusCode OK)	JSON object containing success message.
400 BadRequest	JSON object containing Error message.
500	JSON object containing Error message.

7. <u>Delete book:</u> [Access for only BookRecommender]

Endpoint name: "/api/books /{bookId}"

Method: DELETE

Parameter: bookId

Response:

Status Code	Response body
200 (HttpStatusCode OK)	JSON object containing success message.
404 NotFound	JSON object containing Error message.
500	JSON object containing Error message.

Configure Azure Environment and Application Settings

Step 1: Set Azure Environment Variables

Declare the given Azure Credentials in the cred.sh file.

- AZURE_CLIENT_ID
- AZURE_CLIENT_SECRET
- AZURE_TENANT_ID
- AZURE_SUBSCRIPTION_ID

Set the environment variables for the Azure client by running the following commands:

- cd dotnetapp
- source cred.sh

Step 2: Update **appsettings.json**

In the appsettings.json file, declare the following values to configure Key Vault and JWT settings:

```
{
  "KeyVaultConfiguration": {
    "URL": "replace with your keyvault url",
    "ConnectionStringSecretName": " replace with your connect secret name",
    "BlobSecret": " replace with your blob secret"
  },
  "BlobContainerName": " replace with your blob container name",
  "JWT": {
    "ValidAudience": " replace with your valid audience",
    "ValidIssuer": " replace with your valid issuer",
    "Secret": "replace with your secret"
  }
}
```

Step 3: Implement **DefaultAzureCredential** in program.cs file

Ensure that the entire application uses the **DefaultAzureCredential** method for authentication.

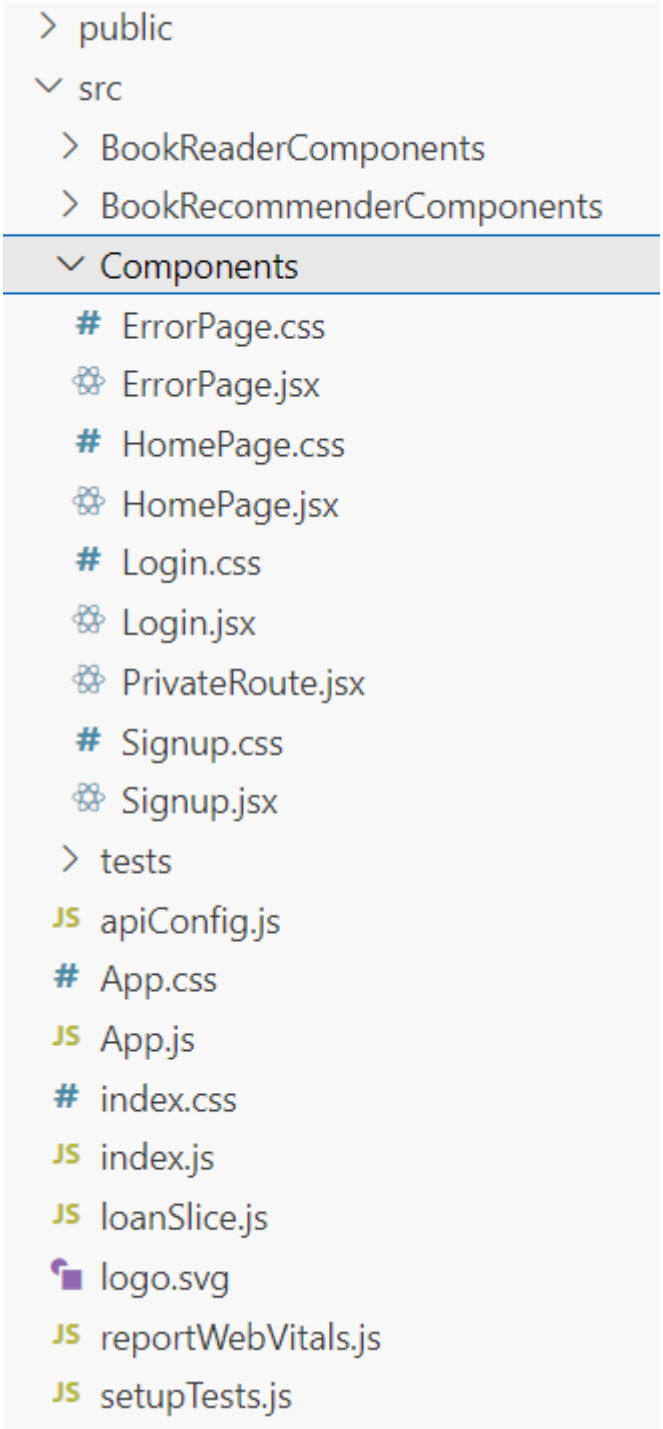
Frontend Requirements:

- Create a folder named Components inside the src to store all common components. (Refer book structure screenshots).
- Create a folder named BookRecommenderComponents inside the src to store all components for BookRecommender. (Refer book structure screenshots).

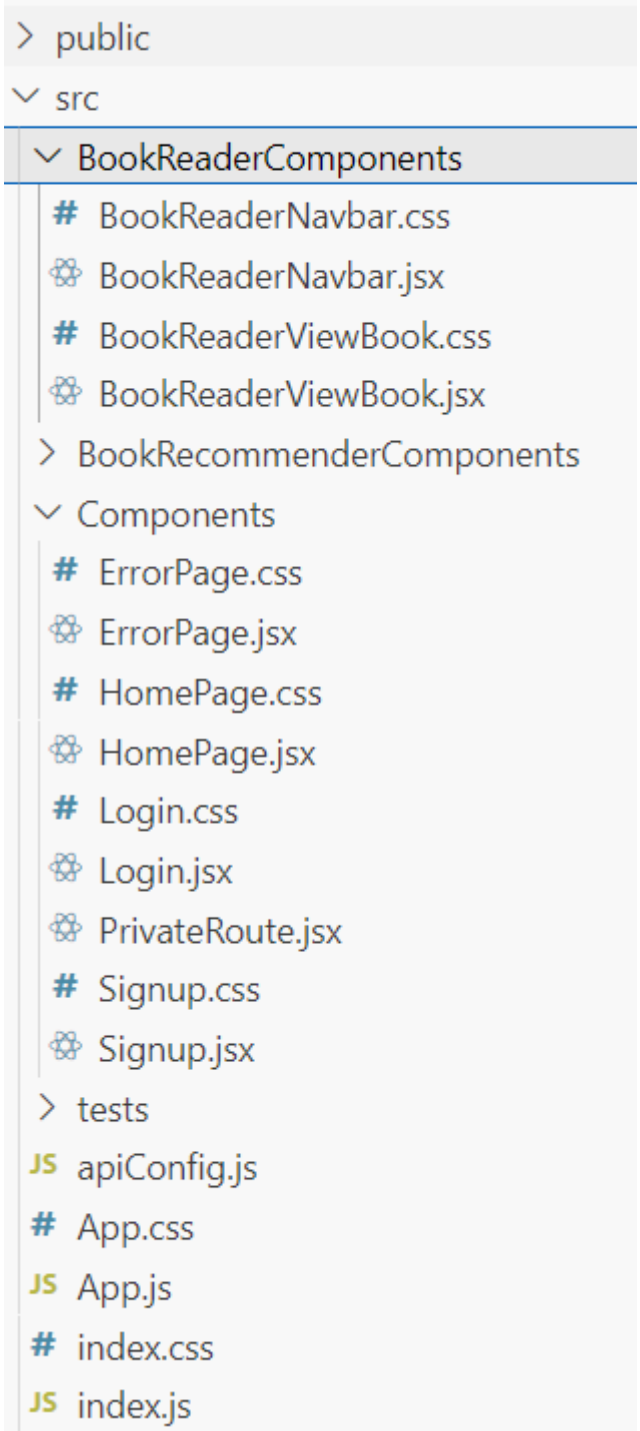
- Create a folder named BookReaderComponents inside the src to store all components for BookReader. (Refer book structure screenshots).
- You can create your own components based on the application requirements.

Book Folder Screenshot:

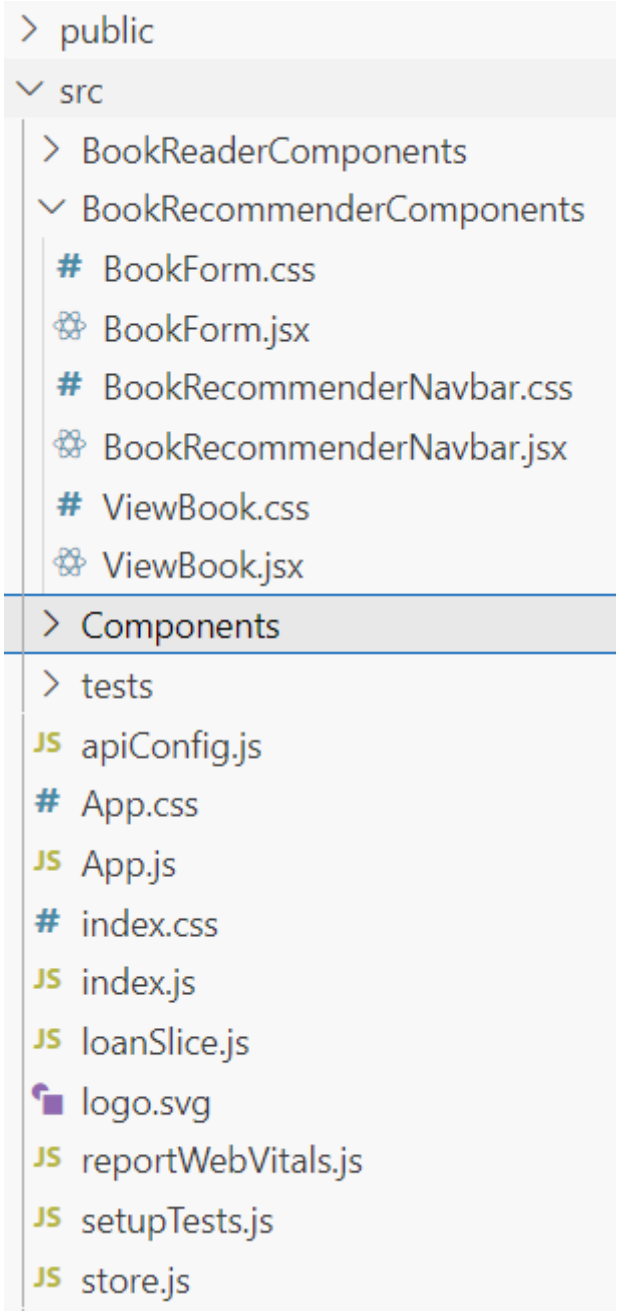
Components:



BookReader Components:



BookRecommenderComponents:



Validations:

Client-Side Validation:

- Implement client-side validation using HTML5 attributes and JavaScript to validate user input before making API requests.
- Provide immediate feedback to users for invalid input, such as displaying error messages near the input fields.

Server-Side Validation:

- Implement server-side validation in the controllers to ensure data integrity.
- Validate user input and API responses to prevent unexpected or malicious data from affecting the application.
- Return appropriate validation error messages to the user interface for any validation failures.

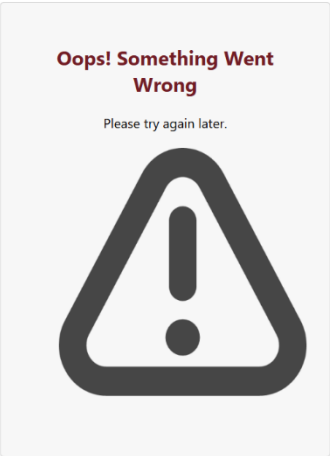
Exception Handling:

- Implement exception handling mechanisms in the controllers to gracefully handle errors and exceptions. Define custom exception classes for different error scenarios, such as API communication errors or database errors.
- Log exceptions for debugging purposes while presenting user-friendly error messages to users. Record all the exceptions and errors handled store in separate table "ErrorLogs".

Error Pages:

Create custom error pages for different HTTP status codes (e.g., 404 Not Found, 500 Internal Server Error) to provide a consistent and user-friendly error experience. Ensure that error pages contain helpful information and guidance for users.

Thus, create a reliable and user-friendly web application that not only meets user expectations but also provides a robust and secure experience, even when faced with unexpected situations. Error page must be displayed if something goes wrong.



Note: Refer to the screenshot for additional details and design the forms accordingly.

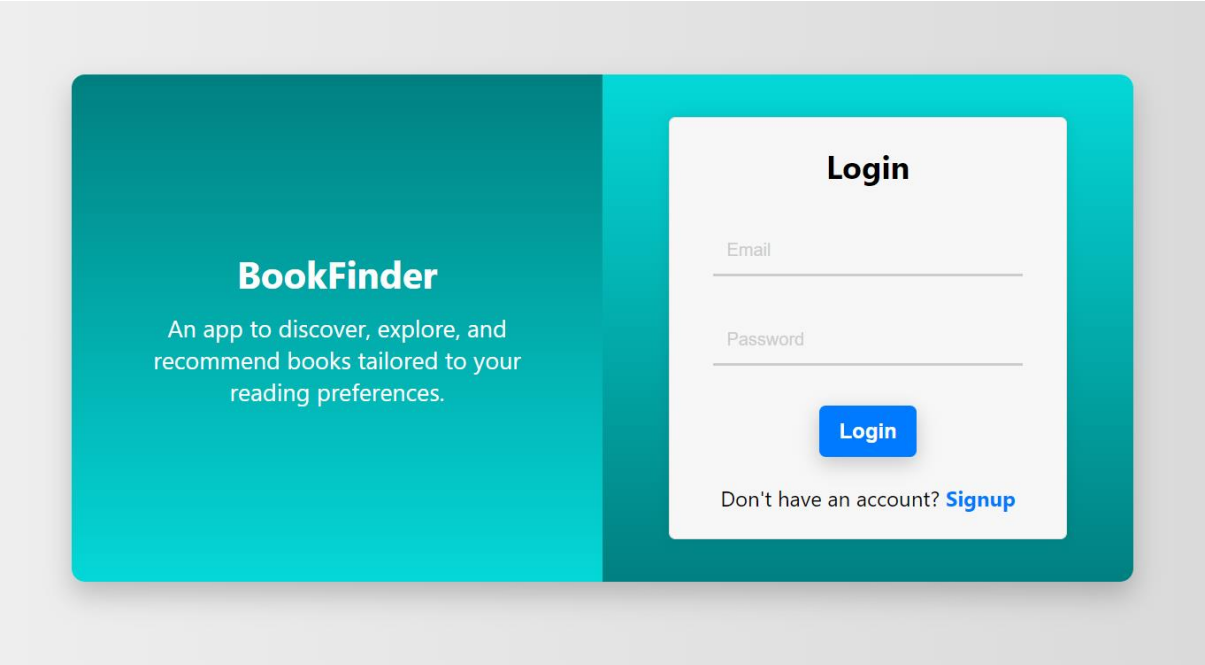
Frontend Screenshots:
• All the asterisk (*) marked fields are mandatory in the form. Make sure to mark all the field names with * symbol followed by the validations.

Navigation Bar:
(BookRecommenderNavbar component)

(BookReaderNavbar component)

Landing Page:

(**Login component**)

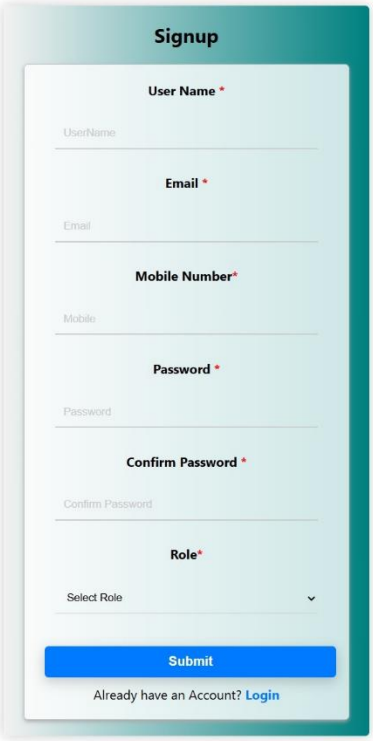


Component features a heading "**BookFinder**" accompanied by an introductory message that provides an overview of the application with login form**.**

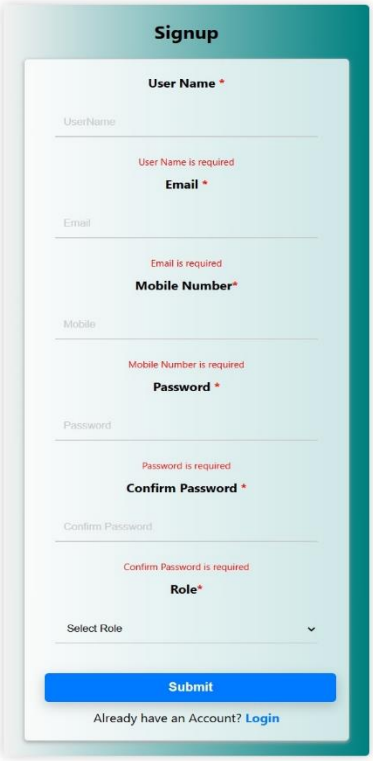
<u>Common Screens (BookRecommender and BookReader)</u>

Signup Page:
(**Signup component**)

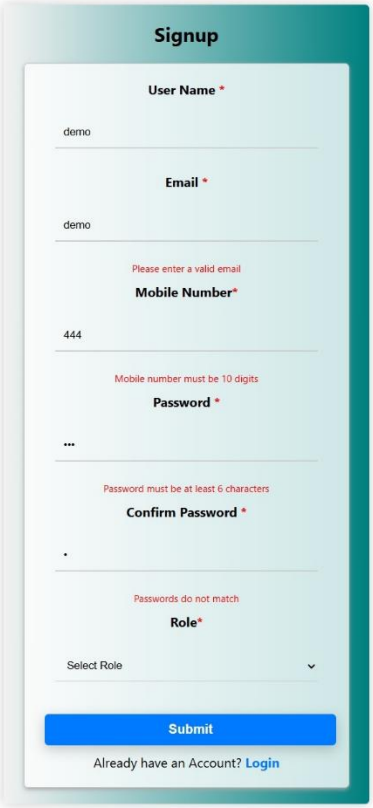
Clicking "**Signup**" in the login page will navigate to the registration page for both roles. Please refer to the screenshots below for validations.

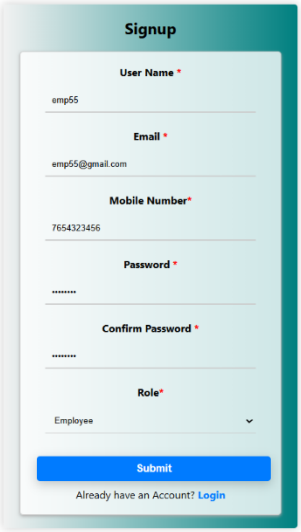


Onclicking “Submit” button with empty field will display the validation message. .

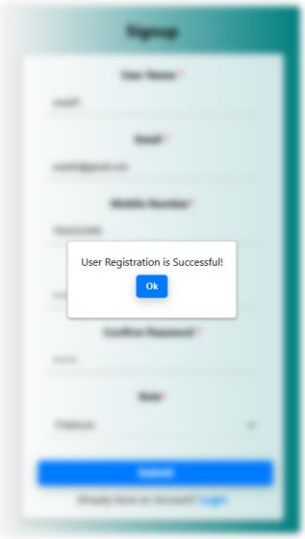


Regular expressions (Regex) are also included to validate email formats, mobile numbers, and to ensure password matching.





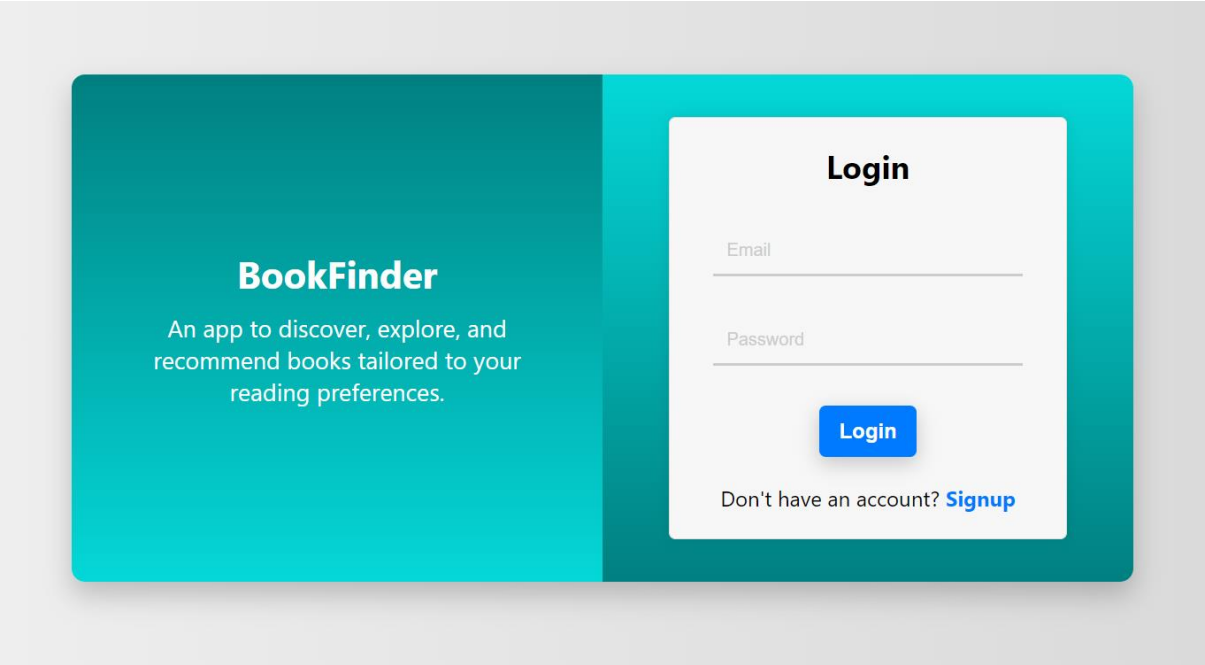
When the "Submit" button is clicked, upon successful submission, popup modal will display.



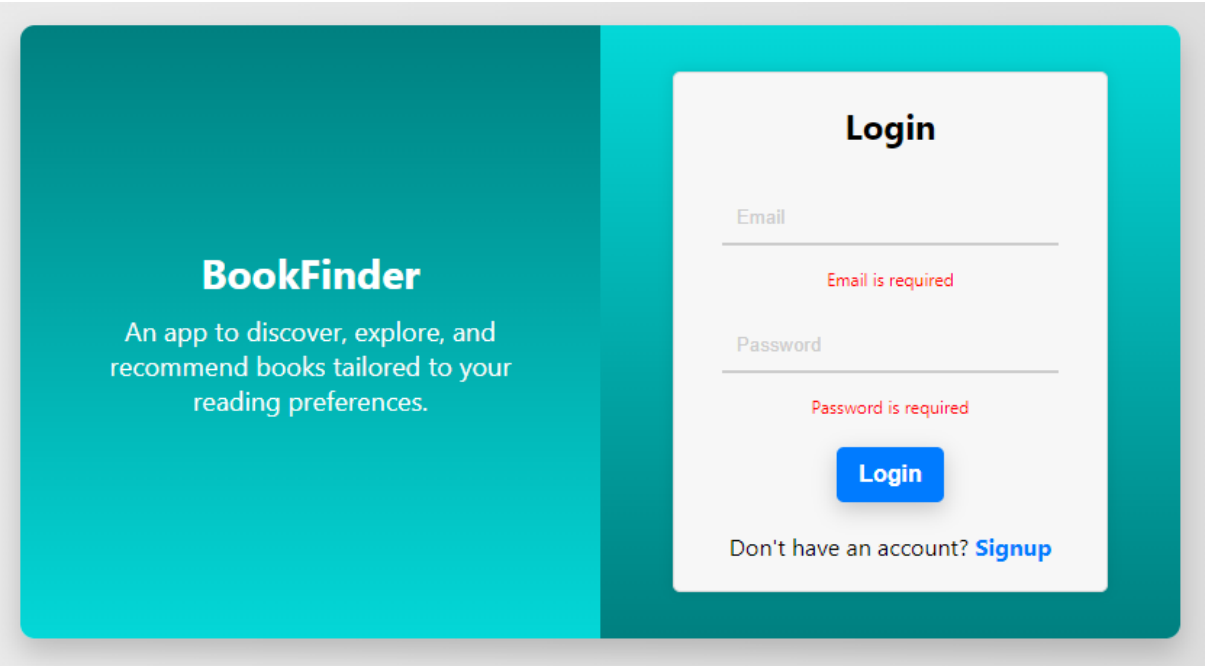
Upon successful completion, a modal will appear, prompting users to click the "Ok" button. Upon clicking this button, users will be directed to the login page, signifying a successful signup process

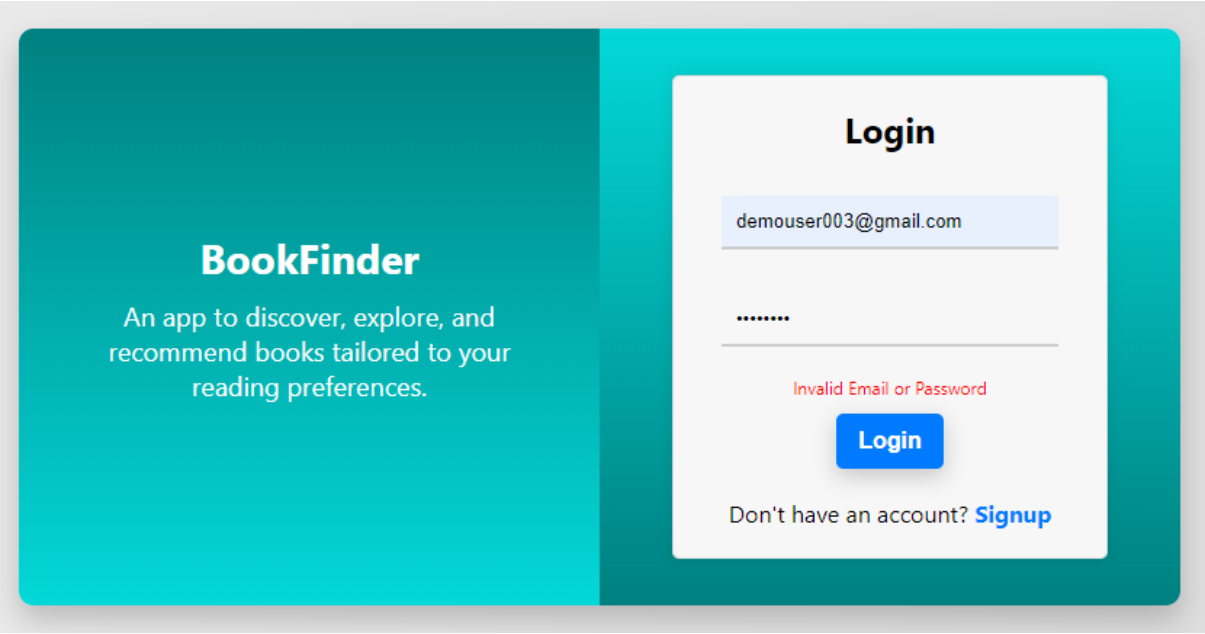
Login Page
(**login component**)

This page is used for logging in to the application. On providing the valid email and password, the user will be logged in.



Perform validations for email and password fields.
On clicking the Login button with empty fields, error messages should be displayed.



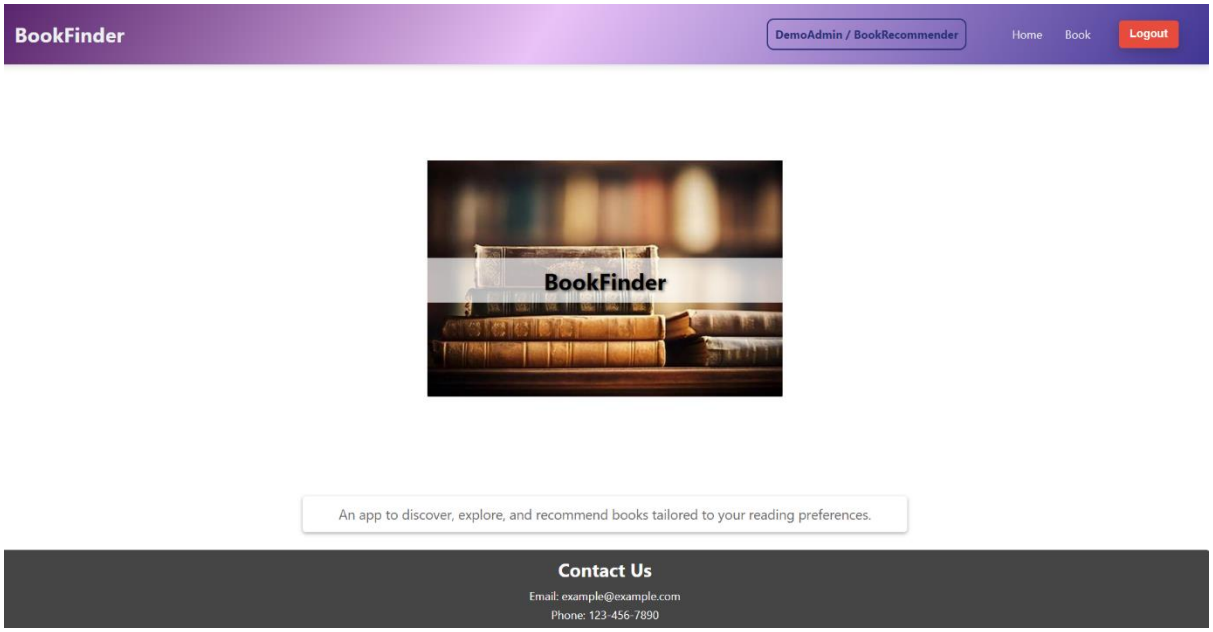


On Clicking the “Login” button, user will be navigated to the (BookRecommenderNavbar or BookReaderNavbar component) based on their roles.

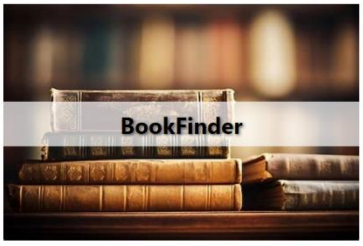
BookRecommender Side:

BookRecommenderNavbar component:

Upon successful login, if the user is a Book Reader, the (BookReaderNavbar component) will be displayed. If the user is a Book Recommender, the (BookRecommenderNavbar component) will be displayed. Additionally, the role-based navigation bar will also display login information such as the username and role.



HomePage Component: This page is used to display the information about BookFinder application. On clicking the '**Home**' tab, user can view the information about the application. (bookfindercoverimage.jpeg provided in public folder)



An app to discover, explore, and recommend books tailored to your reading preferences.

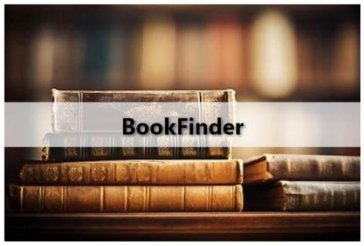
Contact Us

Email: example@example.com

Phone: 123-456-7890

Managers can navigate to other pages by clicking on the menu available in the navigation bar.

Clicking the “Book” option in the navbar will hover “Add Book” and “View Book” options, allowing the BookRecommender to add new book and view existing ones.



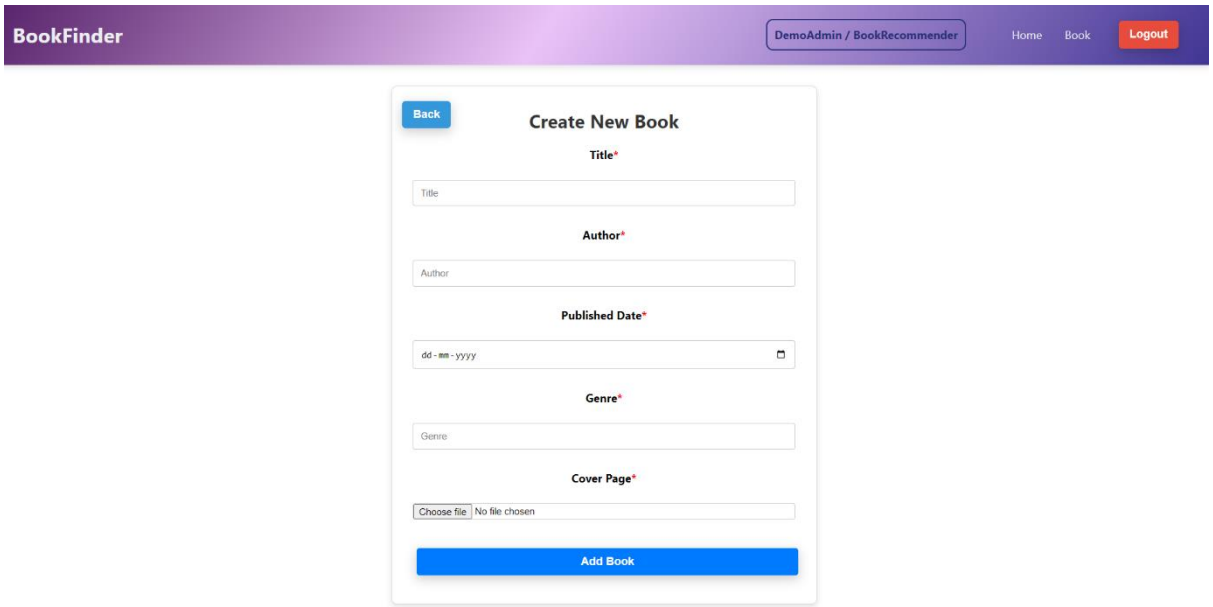
An app to discover, explore, and recommend books tailored to your reading preferences.

Contact Us

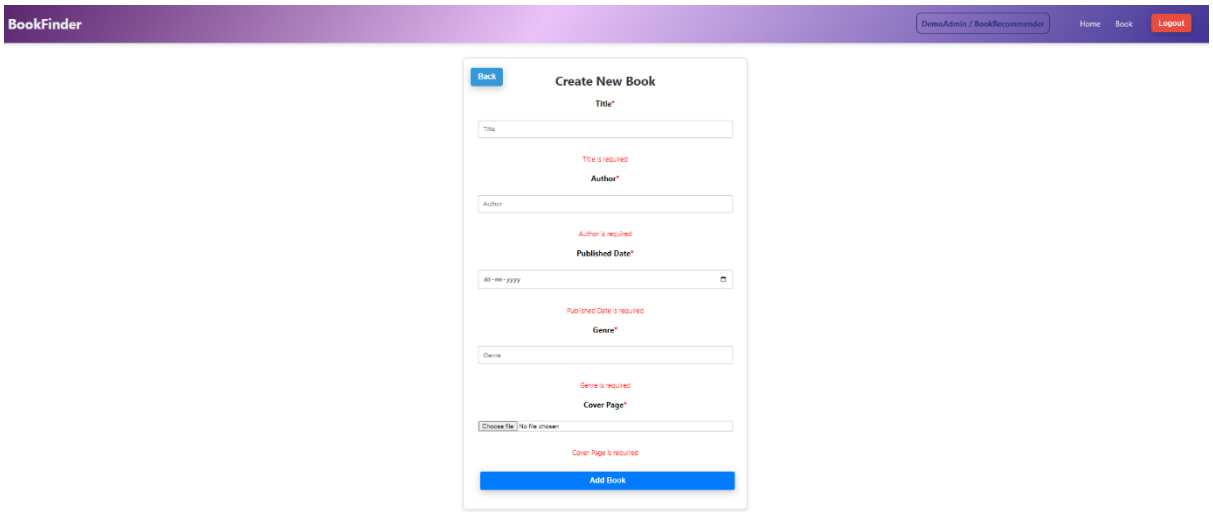
Email: example@example.com

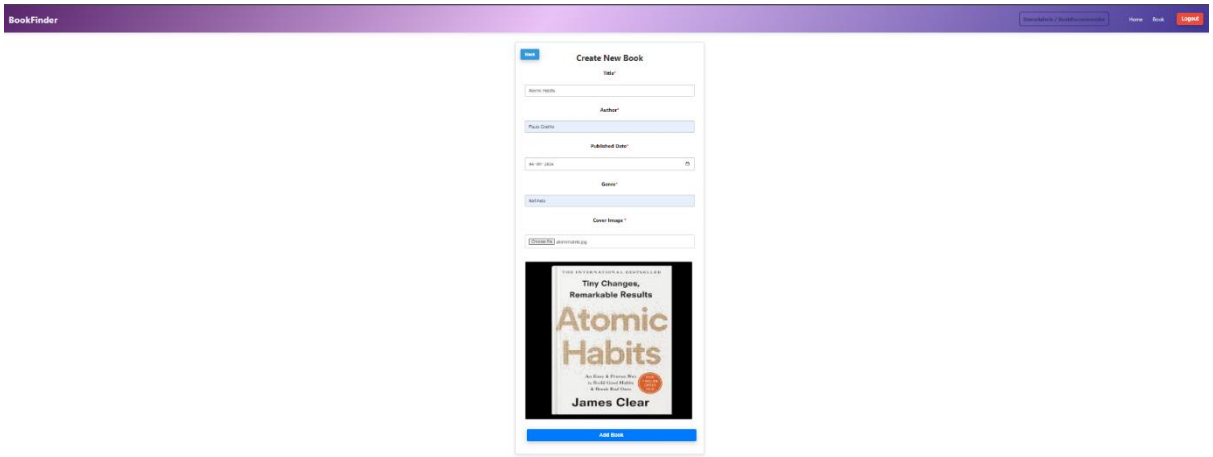
Phone: 123-456-7890

By clicking the “Add Book” option will navigate to the “**BookForm**” component, where the BookRecommender can add Book.

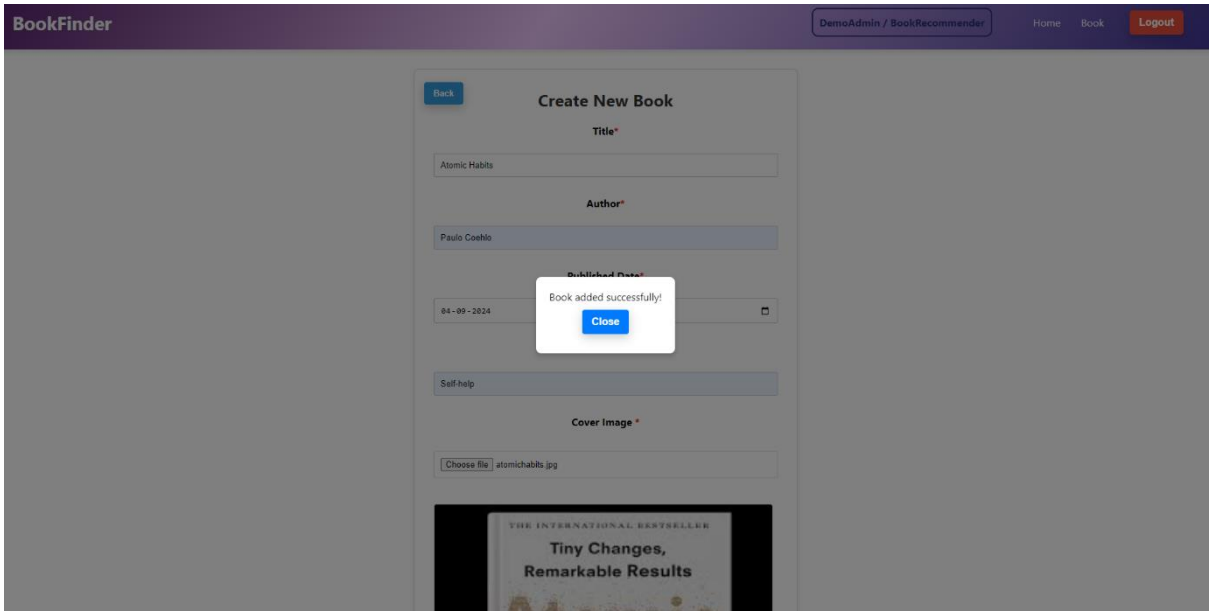


On Clicking “Add Book” button with empty field will display validations error message.

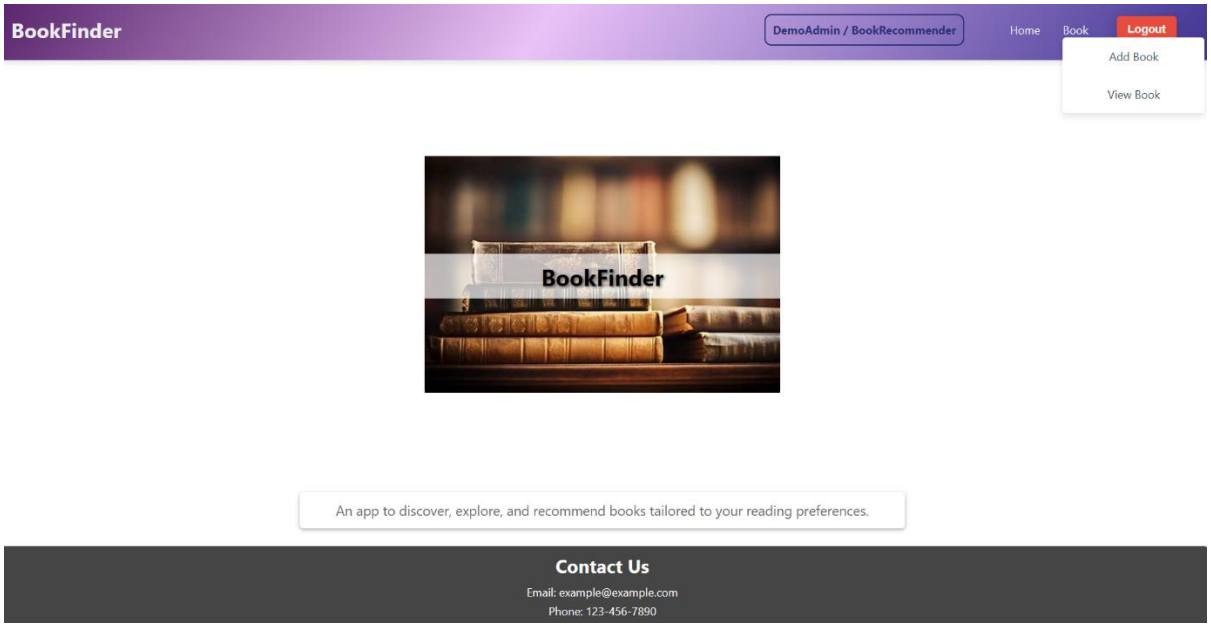




Upon successfully adding a book, a success popup modal with a "Close" button will be display.

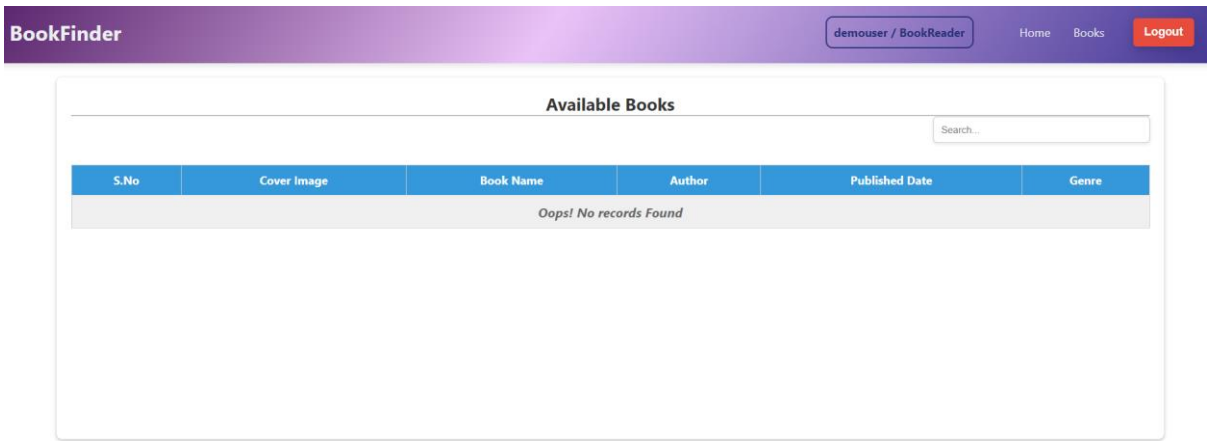


Clicking on “Close” button will navigate to “**ViewBook**” component.

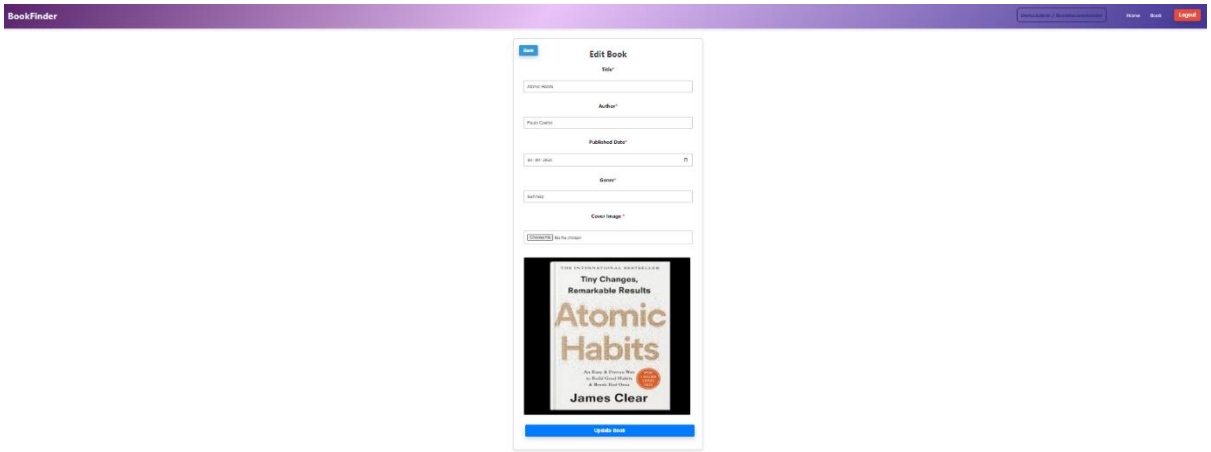


When clicking the “View Book” option under “Book” it navigates to the list of books. It should be in table format.

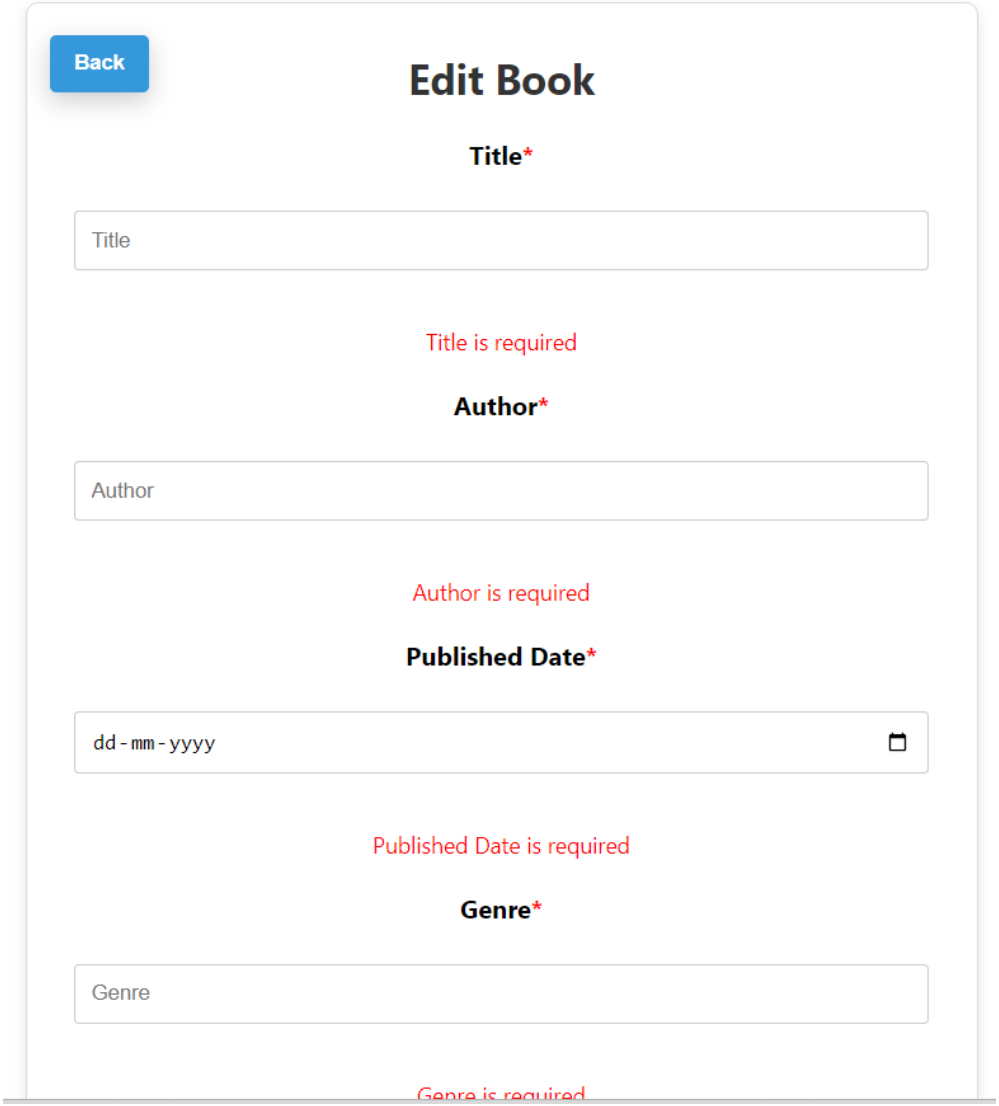
If Data is not available to display, then it will display “Oops! No records Found”.



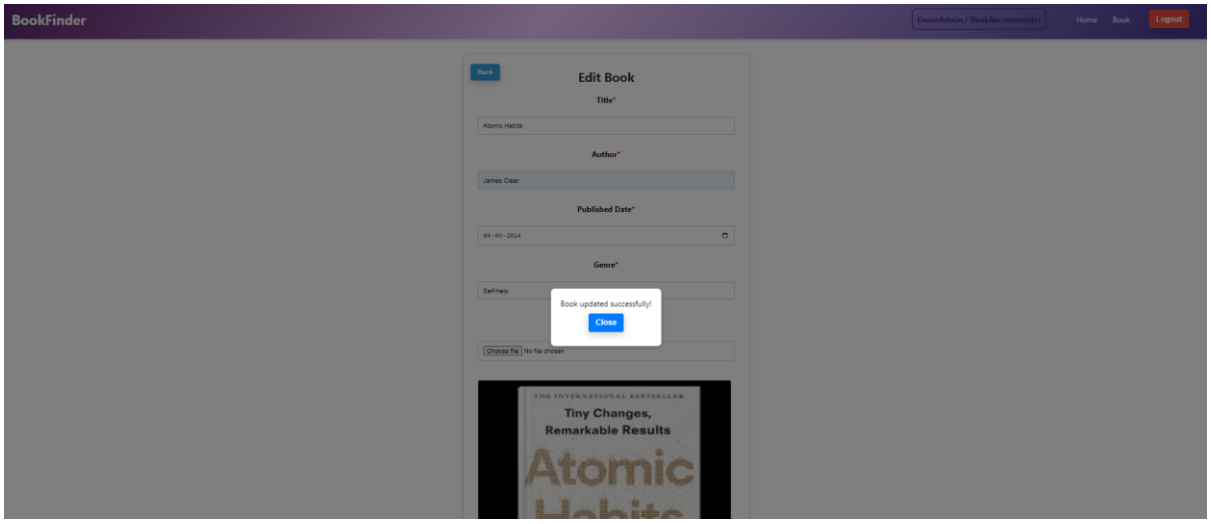
Clicking the "Edit" button will navigate to the BookForm component, displaying the edit form prepopulated with the data of the selected Book.



Validation message is displayed on clicking “Update Book” with empty field.

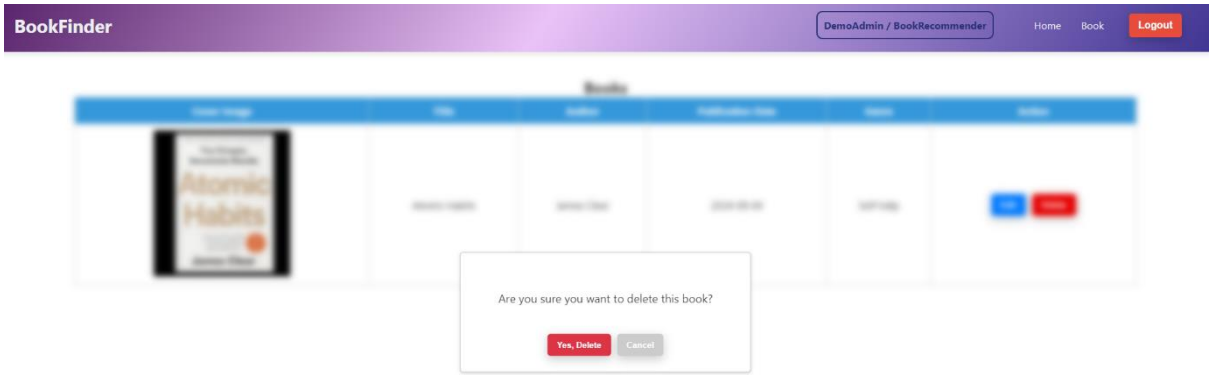


Upon successfully updating a book, a success model with a "Close" button will be displayed.



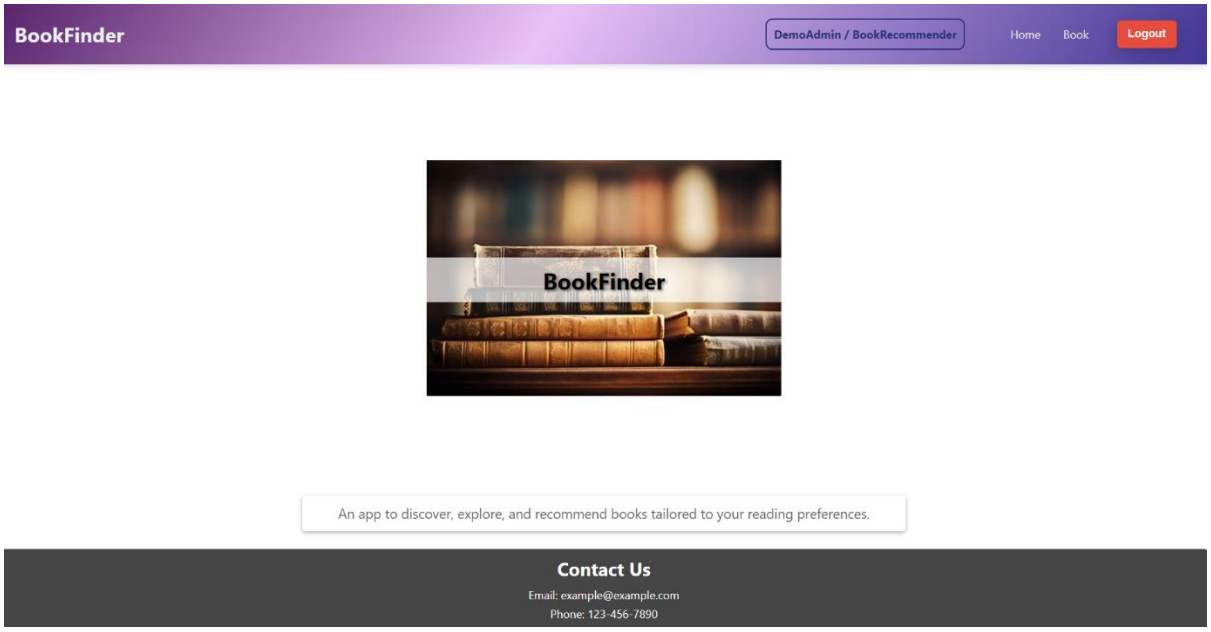
Clicking on “Close” button will navigate to “ViewBook” component

When clicking the "Delete" button, a confirmation modal will be displayed with the message “**Are you sure you want to delete this book?”**

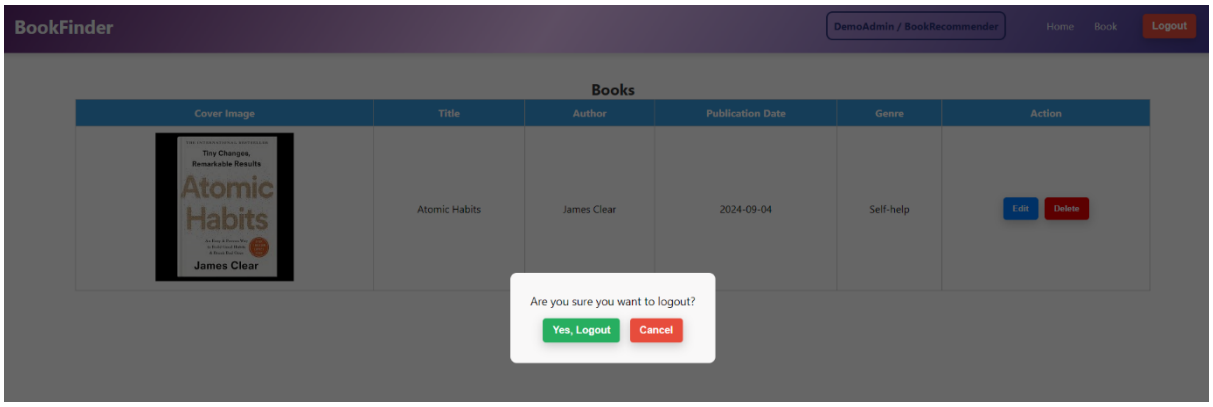


Clicking "Yes, Delete" will remove the selected book and refresh the displayed data.

Clicking “Cancel” will close the popup window.

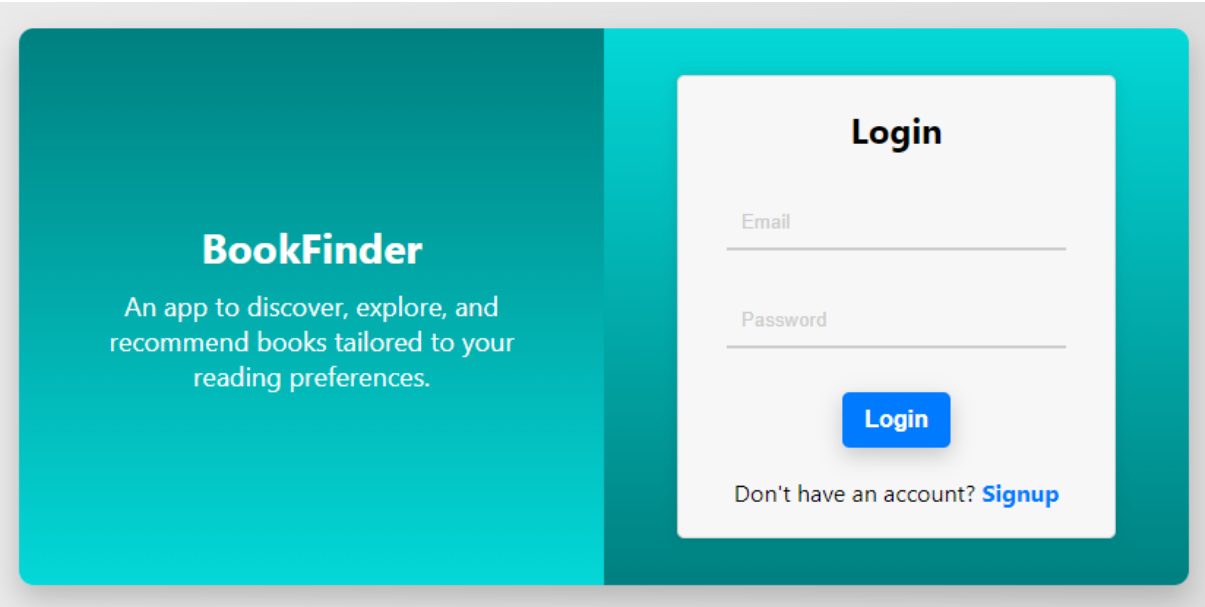


On clicking the "Logout" button, a pop-up should be displayed with confirmatory message to logout the user.



OnClicking “Cancel” will close the modal window displayed.

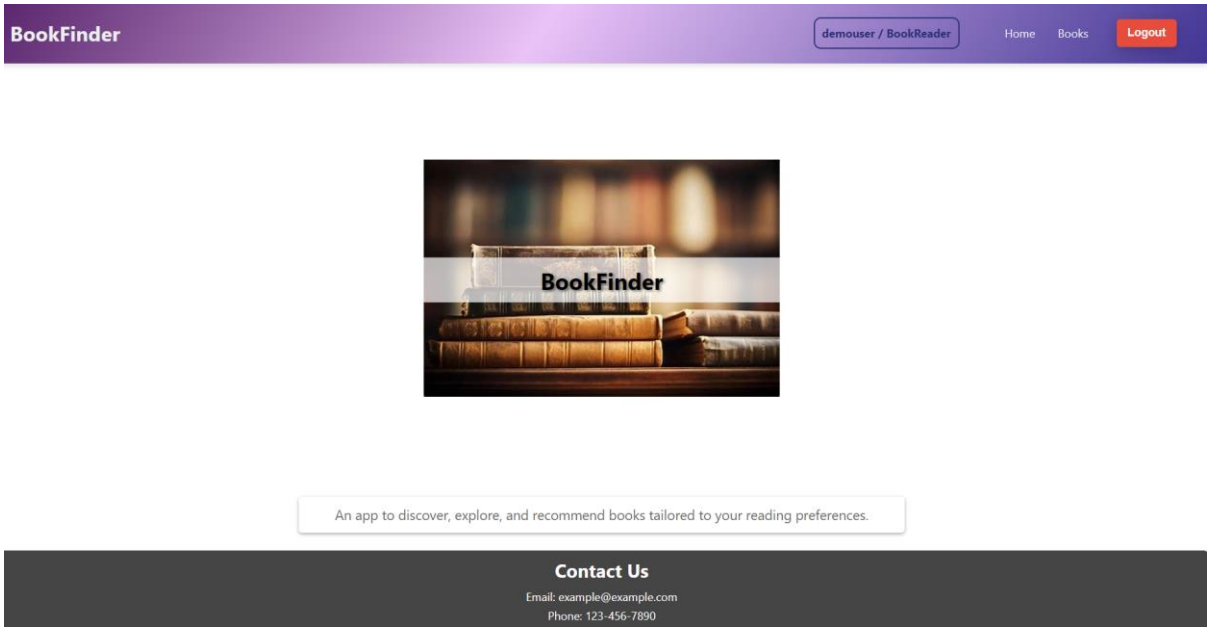
OnClicking "Yes, Logout" will navigate to the login component.



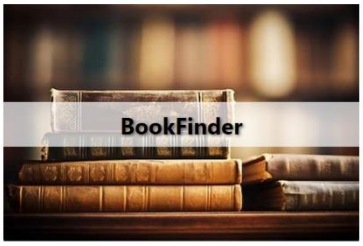
BookReader Side:

BookReaderNavbar component:

Upon successful login, if the user is a book reader, the (BookReaderNavbar component) will be displayed. If the user is a Book Recommender, the (BookRecommenderNavbar component) will be displayed. Additionally, the role-based navigation bar will also display login information such as the username and role.



HomePage Component: This page is used to display the information about BookFinder application. On clicking the '**Home**' tab, user can view the information about the application. (bookfindercoverimage.jpg provided in public folder)



An app to discover, explore, and recommend books tailored to your reading preferences.

Contact Us

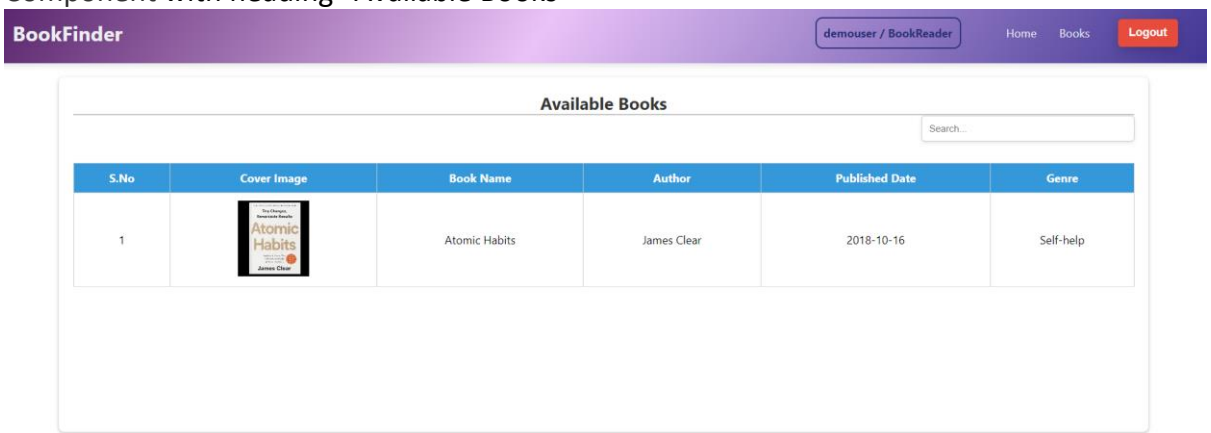
Email: example@example.com

Phone: 123-456-7890

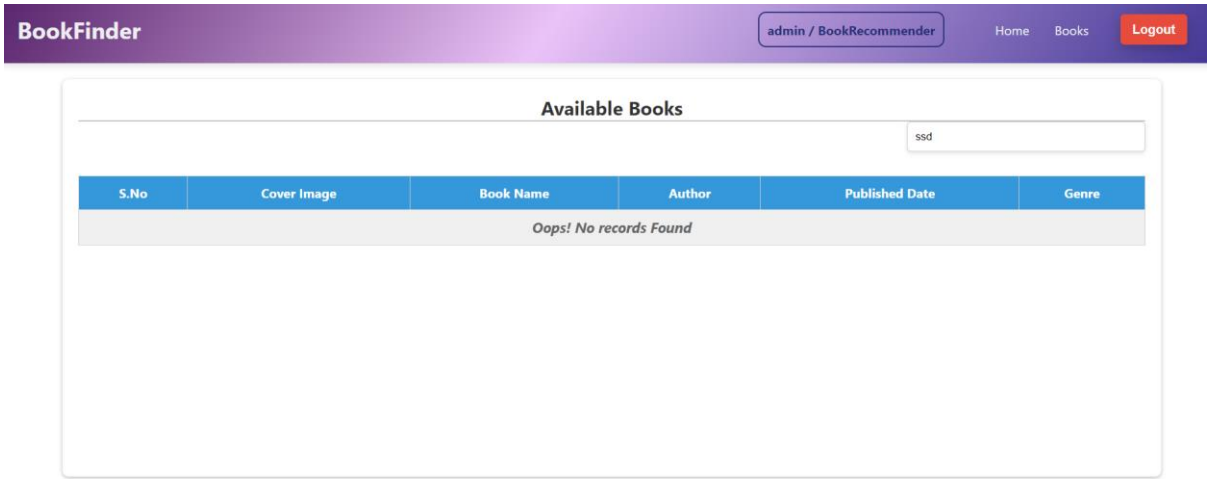
BookReader can navigate to other pages by clicking on the menu available in the navigation bar.

BookReaderViewBook component

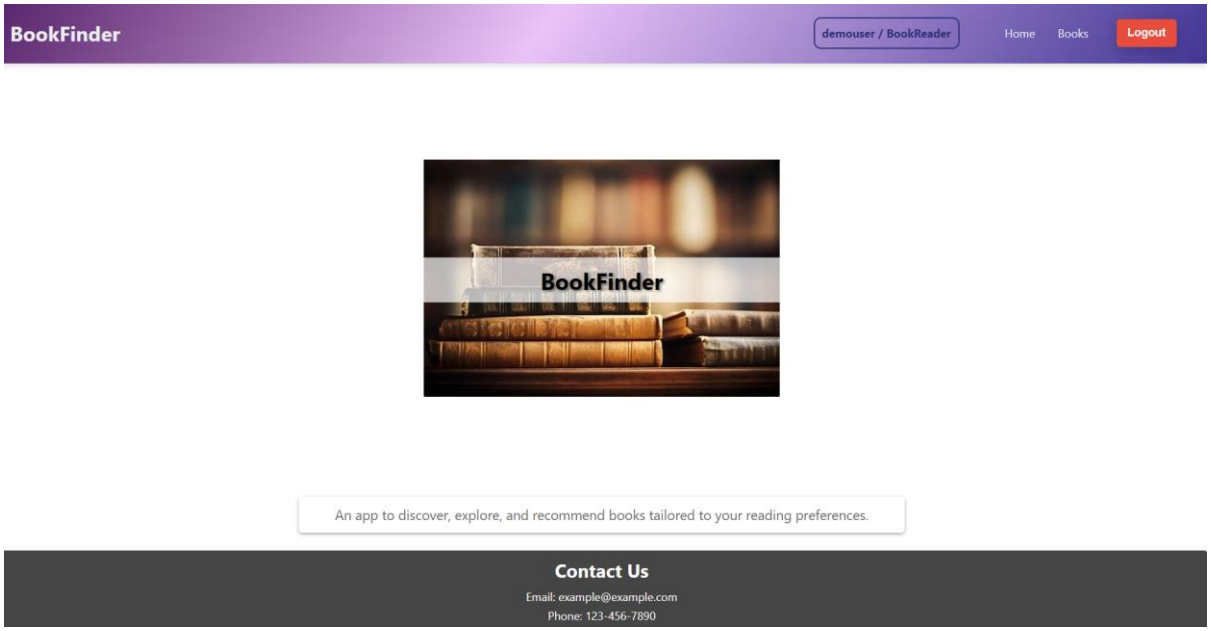
On Clicking over the "Books" item in the navbar, will navigate to "**BookReaderViewBook**" Component with heading "Available Books"



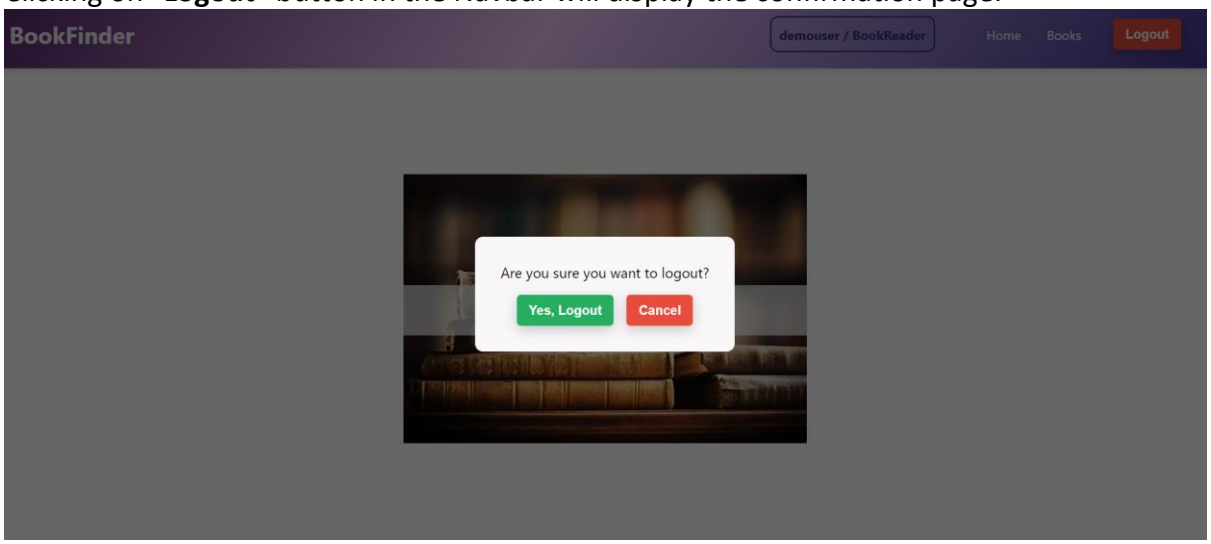
Additionally with search functionality based on book name .



If the searched book is unavailable, display the message: "Oops! No records Found."

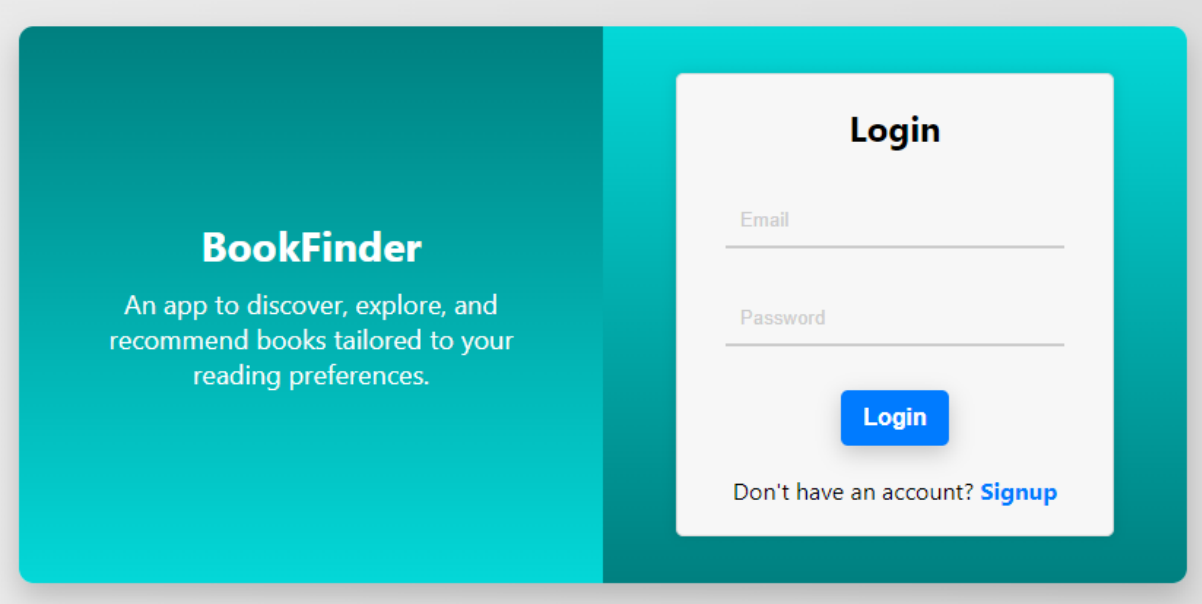


Clicking on "**Logout**" button in the Navbar will display the confirmation page.



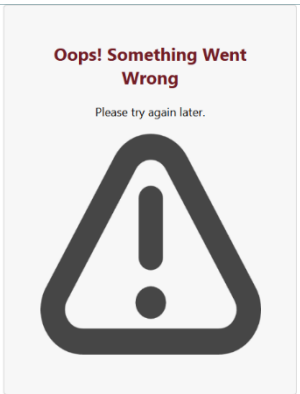
OnClicking "Yes, Logout" will navigate to the login component.

OnClicking “Cancel” will close the modal window displayed.



ErrorPage Component:

This page displays with “Oops! Something Went Wrong” message with the image (alert.png) provided in public folder.



Commands to Run the Project:

cd dotnetapp

Select the dotnet project folder

dotnet tool restore

This command will restore all the required packages to run the application.

dotnet run

To run the application in port 8080 (The settings preloaded click 8080 Port to View)

dotnet build

To build and check for errors

dotnet clean

If the same error persists clean the project and build again

To work with Entity Framework Core:

Install EF using the following commands:

dotnet new tool-manifest

dotnet tool install --local dotnet-ef --version 6.0.6

dotnet dotnet-ef --To check the EF installed or not

dotnet dotnet-ef migrations add "InitialSetup" --command to setup the initial creation of tables mentioned in DBContext

dotnet dotnet-ef database update --command to update the database

To Work with SQLServer:

(Open a New Terminal) type the below commands

sqlcmd -U sa

password: examlyMssql@123

1> create database appdb

2> go

>use appdb

>go

1> create table TableName(id int identity(1,1),.....)

2> go

Note:

The database name should be appdb.

Use the below sample connection string to connect the MsSql Server

private string connectionString = "User ID=sa; password=examlyMssql@123; server=localhost**;** Database=appdb; trusted_connection=false; Persist Security Info=False; Encrypt=False";

Platform Instructions to run the React project:
Step 1:
Use "cd reactapp" command to go inside the reactapp folder
Install Node Modules - "npm install**"**
Step 2:
Write the code inside src folder
Create the necessary components and enter the code
Step 3:
Command to run the project "npm start"
Step 4:
To run the test cases click the "Run Test Case" button.

Note :
Click PORT 8081 to view the result / output.
If any error persists while running the app, delete the node modules and reinstall them.

Cloud Deployment:
1. Introduction
1.1 Scope

The book involves deploying two separate applications (backend and frontend) to Azure App Services, integrating with Azure Key Vault for secrets management, using Azure SQL Database for data storage, and Azure Storage Account for storing uploaded images. The deployment will also include the creation and configuration of a service principal for secure access and necessary firewall rules for the SQL database.

Cloud Deployment

2.1 System Architecture

The system is divided into two main components: Backend and Frontend. Each component uses various Azure services to handle specific tasks.

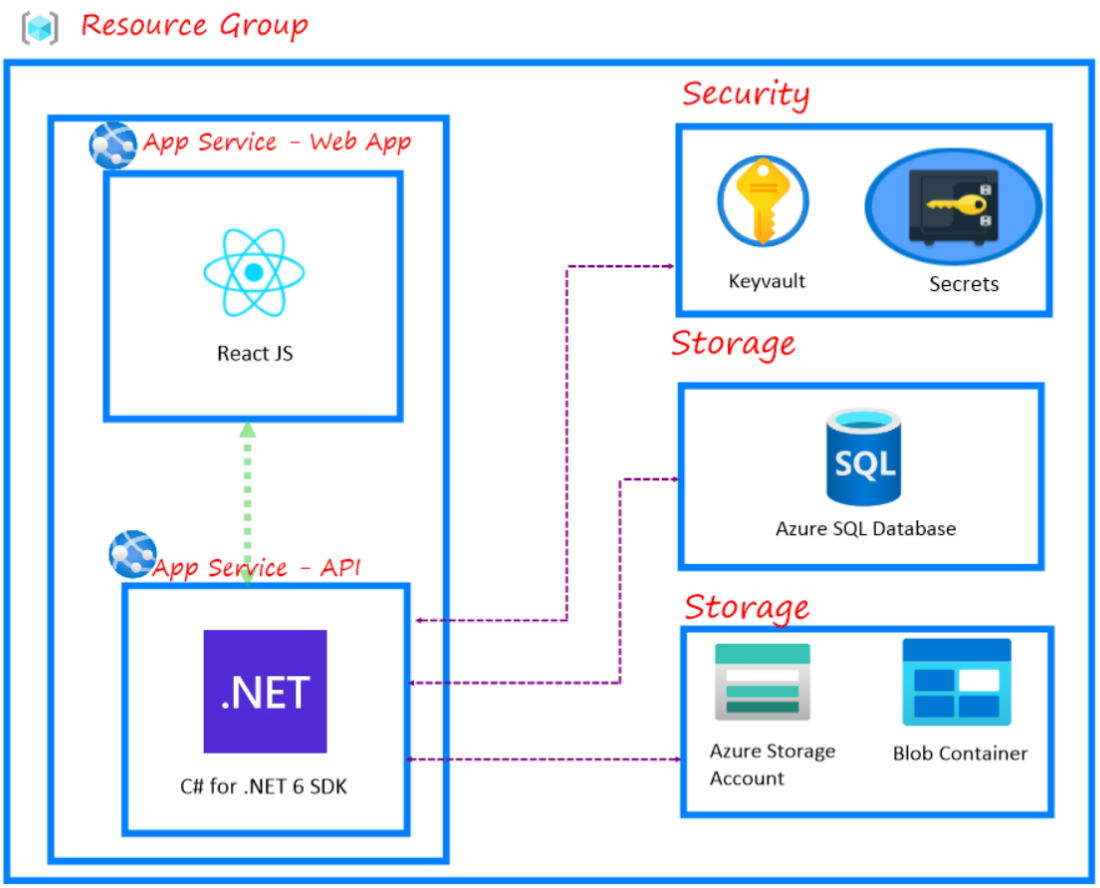
2.2 Backend Architecture

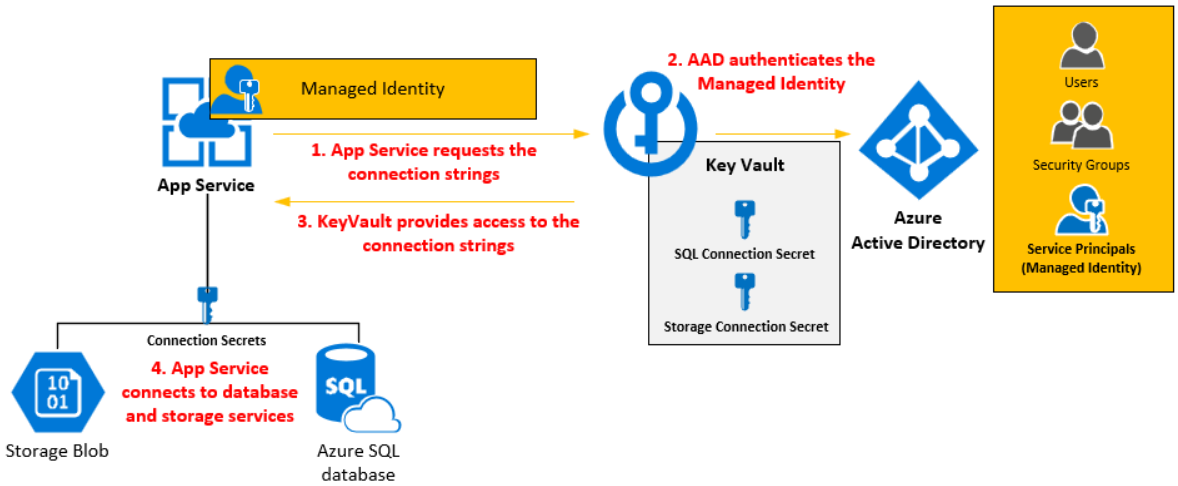
- **Azure App Service for .NET Backend**
  - App Name: "<Resource Group Name>bapp"
  - Configuration:
    - Deploy .NET backend using GitHub Actions
    - Configure to retrieve secrets from Azure Key Vault
    - Connect to Azure SQL Database using connection string from Key Vault
    - Store uploaded images in Azure Storage Account
- **Azure Key Vault**
  - Key Vault Name: <Resource Group Name>
  - Usage:
    - Store database connection strings, storage account keys, and other secrets
- **Azure SQL Database**
  - Database Name: "<Backend App Service Name>-database"
  - Configuration:
    - Apply firewall rules to allow required IP addresses
    - Store connection string in Azure Key Vault
- **Azure Storage Account**
  - Storage Account Name: <Resource Group Name>
  - Configuration:
    - Use blob storage to store uploaded images
    - Store storage account connection string in Azure Key Vault
- **Service Principal**
  - Usage:
    - Created for access management
    - Used in environment variables and GitHub secrets for deployments

2.3 Frontend Architecture

- **Azure App Service for React Frontend**
  - App Name: “<Resource Group Name>fapp”
  - Configuration:
    - Deploy React frontend using GitHub Actions
    - Configure to retrieve secrets from Azure Key Vault

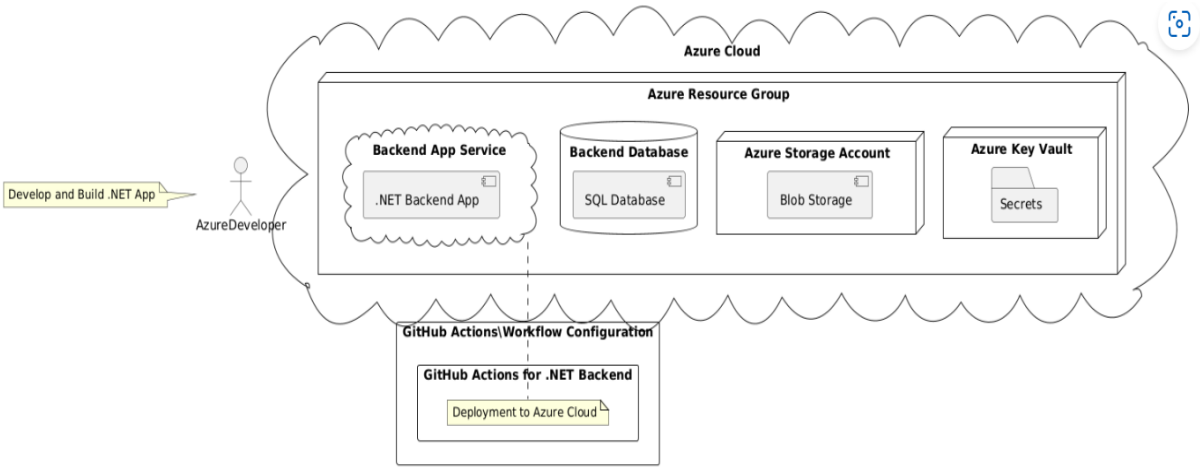
2.4 Architecture Diagram



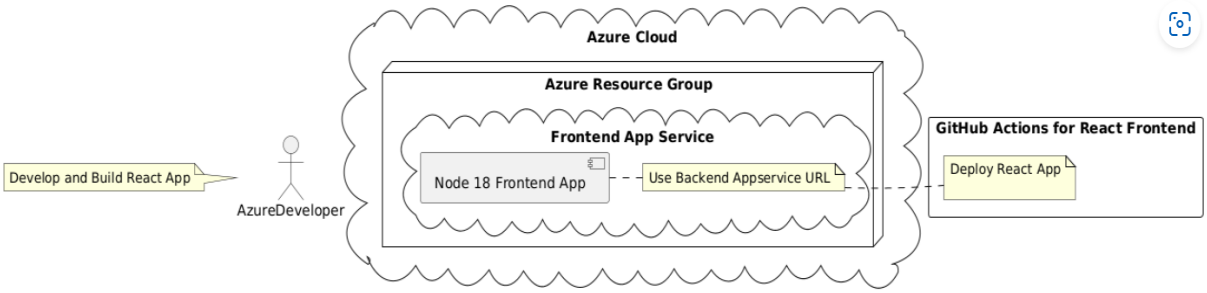


2.5 Flow Diagram

Backend Flow Diagram:



Frontend Flow Diagram:



3. Specific Requirements

3.1 Backend Services

Azure App Service for .NET Backend

- App Name: “<Resource Group Name>bapp”
- Deployment Steps:
  - Configure GitHub Actions workflow for deploying the .NET backend to bendapp24.
  - Run the GitHub Actions workflow.

Azure Key Vault

- Key Vault Name: <Resource Group Name>
- Configuration Steps:

    - Store necessary secrets (e.g., database connection strings, storage account keys) in edhubkvlt24.
    - Update backend application to fetch secrets from Key Vault.

Azure SQL Database

- Database Name: “<Backend App Service Name>-database”
- Configuration Steps:
    - Create an Azure SQL Database.
    - Add firewall rules to allow required IP addresses.
    - Store the connection string in Azure Key Vault.
    - Update the backend application to use the connection string from Key Vault.

Azure Storage Account

- Storage Account Name: <Resource Group Name>
- Configuration Steps:
    - Create an Azure Storage Account with blob storage.
    - Store the storage account connection string in Azure Key Vault.
    - Update the backend application to use the storage account for storing uploaded images.

Service Principal

- Configuration Steps:
    - Create a service principal and assign necessary permissions.
    - Store the service principal credentials as environment variables and GitHub secrets.
    - Update GitHub Actions workflows to use the service principal for deployments.

3.2 Frontend Services

Azure App Service for React Frontend

- App Name: “<Resource Group Name>fapp”
- Deployment Steps:
    - Configure GitHub Actions workflow for deploying the React frontend to “<Resource Group Name>fapp”.
    - Run the GitHub Actions workflow.

3.3 GitHub Actions

GitHub Actions for .NET Backend

- Workflow Configuration:
    - Create a GitHub Actions workflow file to deploy the .NET backend to “<Resource Group Name>bapp”

    - Ensure the workflow includes steps for building the .NET application, logging into Azure, and deploying the application to Azure App Service.

GitHub Actions for React Frontend

- Workflow Configuration:
    - Create a GitHub Actions workflow file to deploy the React frontend to frontendapp24.
    - Ensure the workflow includes steps for building the React application, logging into Azure, and deploying the application to Azure App Service.

4.2 Integration with Azure Key Vault

- Steps:
    1. Store necessary secrets (e.g., database connection strings, storage account keys) in <Resource Group Name>
    2. Update both backend and frontend applications to fetch secrets from Key Vault.
    3. Deploy the applications.
- Expected Result: The applications should be able to retrieve and use secrets from the Key Vault successfully.

4.3 Azure SQL Database Configuration

- Steps:
    1. Create an Azure SQL Database.
    2. Add firewall rules to allow required IP addresses.
    3. Store the connection string in Azure Key Vault.
    4. Update the backend application to use the connection string from Key Vault.
    5. Deploy the backend application.
- Expected Result: The backend application should successfully connect to the SQL database using the connection string from Key Vault, and the SQL database should accept connections from the specified IP addresses.

4.4 Azure Storage Account Configuration

- Steps:
    1. Create an Azure Storage Account with blob storage.
    2. Store the storage account connection string in Azure Key Vault.
    3. Update the backend application to use the storage account for storing uploaded images.
    4. Deploy the backend application.
- Expected Result: The backend application should successfully store and retrieve images from the blob storage.

4.5 Service Principal Configuration

- Steps:

1. Create a service principal and assign necessary permissions.
2. Store the service principal credentials as environment variables and GitHub secrets.
3. Update GitHub Actions workflows to use the service principal for deployments.

- Expected Result: The service principal should be successfully used for deployments and accessing resources, ensuring secure and seamless deployment.

General Instructions:

- You will be provided with an Azure Account Login Username and Password
- In the same account, a service principal's Client ID, Client Secret, and that Azure account's Subscription ID, Tenant ID will be provided.

Steps to be followed:

1. Create an Azure SQL database, a Storage Account, and a Keyvault

2. Complete the Backend Dotnet Development by mentioning the Azure SQL Database connection string and Azure Storage Account connection string along with the JWT Secret, JWT Valid issuer, JWT Valid Audience in the appsettings.json file. These connection strings and JWT Credentials totally 5 secrets must be stored in Azure Keyvault as secrets as mentioned below and it shouldn't be disclosed or called upon anywhere in the application development.
    - ASPNETCORE_ENVIRONMENT : Development
    - KeyVaultConfiguration__BlobSecret : blobsecret
    - KeyVaultConfiguration__ConnectionStringSecretName: sqlconnectionstring
    - KeyVaultConfiguration__JWTSecret : JWTSecret
    - KeyVaultConfiguration__JWTValidAudience : JWTValidAudience
    - KeyVaultConfiguration__URL : https://<Your Keyvault Name>.vault.azure.net

3. Once completed the Dotnet Backend Development, complete the Frontend React Development

4. Do the platform testing in 8080 and 8081 for backend and frontend respectively

5. Deploy two Azure App services for backend and frontend

6. In the Backend Dotnet application Azure App service, mention the Keyvault URL, all the 5 Keyvault secrets, also additional environmental variable 'ASPNETCORE' with the value 'Development' totally 7 are to be given as Azure App settings Environmental Variables.

7. Complete the Backend Deployment to Azure App service via GitHub Actions which can use the Backend Publish Profile as Azure Web App Publish Profile can be used. So this should be stored in GitHub Secrets.

8. After the successful deployment of Dotnet backend application to respective Azure App service, build the Front end React application with the Azure Backend App service URL

9. Once completed the build process successfully, push the Frontend code with 'build' folder to GitHub.

10. Deploy the Frontend application with the already built 'build' folder to the second Azure App service which will be in reactapp build folder.

11. In the deployed Front end React Azure App service, mention in the App settings Environmental variable wih the following Backend App Service URL:

    - API_BASE_URL: "https://<Your Backend App Service Name>.azurewebsites.net"

12. For the React application site access, give the following command in the Frontend App Service Configuration App Settings Start up command :

    pm2 serve /home/site/wwwroot --no-daemon --spa